

表 6-4 几种最主要的简单类型

类型	大小	说明
INTEGER	4 字节	在 -2^{31} 到 $2^{31}-1$ 之间的整数
Integer32	4 字节	和 INTEGER 相同
Unsigned32	4 字节	在 0 到 $2^{32}-1$ 之间的无符号数
OCTET STRING	可变	不超过 65535 字节长的字节串
OBJECT IDENTIFIER	可变	对象标识符
IPAddress	4 字节	由 4 个整数组成的 IP 地址
Counter32	4 字节	可从 0 增加到 2^{32} 的整数；当它到达最大值时就返回到 0
TimeTicks	4 字节	记录时间的计数值，以 1/100 秒为单位
BITS	—	比特串
Opaque	可变	不解释的串

SMI 定义了两种结构化数据类型，即 sequence 和 sequence of。

数据类型 sequence 类似于 C 语言中的 struct 或 record，它是一些简单数据类型的组合（不一定要相同的类型）。而数据类型 sequence of 类似于 C 语言中的 array，它是同样类型的简单数据类型的组合，或同样类型的 sequence 数据类型的组合。

3. 编码方法

SMI 使用 ASN.1 制定的基本编码规则 BER (Basic Encoding Rule) 进行数据的编码。BER 指明了每种数据的类型和值。在发送端用 BER 编码，可把用 ASN.1 所表述的报文转换成唯一的比特序列。在接收端用 BER 进行解码，就可得到该比特序列所表示的 ASN.1 报文。

初看起来，或许用两个字段就能表示类型和值。但由于表示值可能需要多个字节，因此还需要一个指出“要用多少字节表示值”的长度字段。因此 ASN.1 把所有的数据元素都表示为 T-L-V 三个字段组成的序列（见图 6-22）。T 字段(Tag)定义数据的类型，L 字段(Length)定义 V 字段的长度，而 V 字段(Value)定义数据的值。



图 6-22 用 TLV 方法进行编码

(1) T 字段又叫作标记字段，占 1 字节。T 字段比较复杂，因为它要定义的数据类型较多。T 字段又再分为以下三个子字段：

- **类别**（2 位）共四种：通用类(00)，即 ASN.1 定义的类型；应用类(01)，即 SMI 定义的类型；上下文类(10)，即上下文所定义的类型；专用类(11)，保留为特定厂商定义的类型。
- **格式**（1 位）共两种，指出数据类型的种类：简单数据类型(0)，结构化数据类型(1)。

- **编号**（5 位）用来标志不同的数据类型。编号的范围一般为 0 ~ 30。当编号大于 30 时，T 字段就要扩展为多个字节（这种情况很少用到，可参考 ITU-T X.209，这里从略）。

表 6-5 是一些数据类型的 T 字段的编码。

表 6-5 几种数据类型的 T 字段编码

数据类型	类别	格式	编号	T 字段（二进制）	T 字段（十六进制）
INTEGER	00	0	00010	00000010	02
OCTET STRING	00	0	00100	00000100	04
OBJECT IDENTIFIER	00	0	00110	00000110	06
NULL	00	0	00101	00000101	05
Sequence, sequence of	00	1	10000	00110000	30
IPAddress	01	0	00000	01000000	40
Counter	01	0	00001	01000001	41
Gauge	01	0	00010	01000010	42
TimeTicks	01	0	00011	01000011	43
Opaque	01	0	00100	01000100	44

(2) **L 字段**又叫作**长度字段**（单字节或多字节）。当 L 字段为单字节时，其最高位为 0，后面的 7 位定义 V 字段的长度。当 L 字段为多个字节时，其最高位为 1，而后面的 7 位定义后续字节的字节数（用二进制整数表示）。这时，所有的后续字节并置起来的二进制整数定义 V 字段的长度。图 6-23 给出了 L 字段的格式举例。

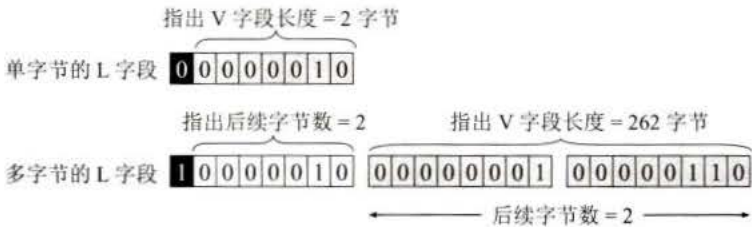


图 6-23 L 字段的格式举例

(3) **V 字段**又叫作**值字段**，用于定义数据元素的值。

根据以上所述，我们给出两个用十六进制表示的编码例子。例如，INTEGER 15，根据表 6-5，其 T 字段是 02，再根据表 6-4，INTEGER 类型要用 4 字节编码。最后得出 TLV 编码为 02 04 00 00 00 0F。又如 IPAddress 192.1.2.3，IPAddress 的 T 字段是 40，V 字段需要 4 字节表示，因此 IPAddress 192.1.2.3 的 TLV 编码是 40 04 C0 01 02 03。

TLV 方法中的 V 字段还可嵌套其他数据元素的 TLV 字段，并可多重嵌套。

6.7.3 管理信息库 MIB

所谓“**管理信息**”就是指在互联网的网管框架中**被管对象的集合**。被管对象必须维持可供管理程序读写的若干控制和状态信息。这些被管对象构成了一个虚拟的信息存储器，所以才称为**管理信息库 MIB**。管理程序就使用 MIB 中这些信息的**值**对网络进行管理（如读取或重新设置这些值）。只有在 MIB 中的对象才是 SNMP 所能够管理的。例如，路由器应当

维持各网络接口的状态、入分组和出分组的流量、丢弃的分组和有差错的报文的统计信息，而调制解调器则应当维持发送和接收的字符数、码元传输速率和接受的呼叫等统计信息。因此在 MIB 中就必须有上面这样一些信息。

我们再看下图 6-21，可以找到节点 mib-2 下面的部分是 MIB 子树。表 6-6 给出了节点 mib-2 所包含的前八个信息类别代表的意思（在后面还有好几个类别）。

表 6-6 节点 mib-2 所包含的信息类别举例

类别	标号	所包含的信息
system	(1)	主机或路由器的操作系统
interfaces	(2)	各种网络接口
address translation	(3)	地址转换（例如，ARP 映射）
ip	(4)	IP 软件
icmp	(5)	ICMP 软件
tcp	(6)	TCP 软件
udp	(7)	UDP 软件
egp	(8)	EGP 软件

我们可以用个简单例子进一步说明 MIB 的意义。例如，从图 6-21 可以看出，对象 ip 的标号是 4。因此，所有与 IP 有关的对象都从前缀 1.3.6.1.2.1.4 开始。

(1) 在节点 ip 下面有个名为 ipInReceives 的 MIB 变量（见图 6-21），表示收到的 IP 数据报数。这个变量的标号是 3，变量的名字是：iso.org.dod.internet.mgmt.mib.ip.ipInReceives，而相应的数值表示是：1.3.6.1.2.1.4.3。

(2) 当 SNMP 在报文中使用 MIB 变量时，对于简单类型的变量，后缀 0 指具有该名字的变量的实例。因此，当这个变量出现在发送给路由器的报文中时，ipInReceives 的数值表示（即变量的一个实例）就是：1.3.6.1.2.1.4.3.0。

(3) 请注意，对于分配给一个 MIB 变量的数值或后缀是完全没有办法进行推算的，必须查找已发布的标准。

上面所说的 MIB 对象命名树的大小并没有限制。下面给出若干 MIB 变量的例子（见表 6-7），以便更好地理解 MIB 的意义。这里的“变量”是指特定对象的一个实例。

表 6-7 MIB 变量的例子

MIB 变量	所属类别	意义
sysUpTime	system	距上次重新启动的时间
ifNumber	interfaces	网络接口数
ifMtu	interfaces	特定接口的最大传送单元 MTU
ipDefaultTTL	ip	IP 在生存时间字段中使用的值
ipInReceives	ip	接收到的数据报数目
ipForwDatagrams	ip	转发的数据报数目
ipOutNoRoutes	ip	路由选择失败的数目
ipReasmOKs	ip	重装的数据报数目
ipFragOKs	ip	分片的数据报数目
ipRoutingTable	ip	IP 路由表
icmpInEchos	icmp	收到的 ICMP 回送请求数目

MIB 变量	所属类别	意义
tcpRtoMin	tcp	TCP 允许的最小重传时间
tcpMaxConn	tcp	允许的最大 TCP 连接数目
tcpInSegs	tcp	已收到的 TCP 报文段数目
udpInDatagrams	udp	已收到的 UDP 数据报数目

上面列举的大多数项目的值可用一个整数来表示。但 MIB 也定义了更复杂的结构。例如, MIB 变量 `ipRoutingTable` 则定义一个完整的路由表。还有其他一些 MIB 变量定义了路由表项目的内容, 并允许网络管理协议访问路由器中的单个项目, 包括前缀、地址掩码以及下一跳地址等。当然, MIB 变量只给出了每个数据项的逻辑定义, 而一个路由器使用的内部数据结构可能与 MIB 的定义不同。当一个查询到达路由器时, 路由器上的代理软件负责 MIB 变量和路由器用于存储信息的数据结构之间的映射。

6.7.4 SNMP 的协议数据单元和报文

实际上, SNMP 的操作只有两种基本的管理功能, 即:

- (1) “读”操作, 用 Get 报文来检测各被管对象的状况;
- (2) “写”操作, 用 Set 报文来改变各被管对象的状况。

SNMP 的这些功能通过探测操作来实现, 即 SNMP 管理进程定时向被管理设备周期性地发送探测信息。上述时间间隔可通过 SNMP 的管理信息库 MIB 来建立。探测的好处是: 第一, 可使系统相对简单; 第二, 能限制通过网络所产生的管理信息的通信量。但探测管理协议不够灵活, 而且所能管理的设备数目不能太多。探测系统的开销也较大。如探测频繁而并未得到有用的报告, 则通信线路和计算机的 CPU 周期就被浪费了。

但 SNMP 不是完全的探测协议, 它允许不经过询问就能发送某些信息。这种信息称为陷阱(trap), 表示它能够捕捉“事件”。但这种陷阱信息的参数是受限制的。

当被管对象的代理检测到有事件发生时, 就检查其门限值。代理只向管理进程报告达到某些门限值的事件(这就叫作过滤)。这种方法的好处是: 第一, 仅在严重事件发生时才发送陷阱; 第二, 陷阱信息很简单且所需字节数很少。

总之, 使用探测(至少是周期性地)以维持对网络资源的实时监视, 同时也采用陷阱机制报告特殊事件, 使得 SNMP 成为一种有效的网络管理协议。

SNMP 使用无连接的 UDP, 因此在网络上传送 SNMP 报文的开销较小。但 UDP 是不保证可靠交付的。这里还要指出, SNMP 使用 UDP 的方法有些特殊。在运行代理程序的服务器端用熟知端口 161 来接收 Get 或 Set 报文和发送响应报文(与熟知端口通信的客户端使用临时端口), 但运行管理程序的客户端则使用熟知端口 162 来接收来自各代理的 trap 报文。

SNMP 现在共定义了如表 6-8 所示的 8 种类型的协议数据单元[RFC 3416, STD62], 其中 PDU 编号为 4 的已经废弃了。在 PDU 编号后面是对应的 T 字段值(十六进制形式表示)。

表 6-8 SNMP 定义的协议数据单元类型

PDU 编号 (T 字段)	PDU 名称	用途
0 (A0)	GetRequest	管理者从代理读取一个或一组变量的值

续表

PDU 编号 (T 字段)	PDU 名称	用途
1 (A1)	GetNextRequest	管理者从代理读取 MIB 树上的下一个变量的值 (即使不知道此变量名也行)。此操作可反复进行, 特别是按顺序一一读取列表中的值很方便
2 (A2)	Response	代理向管理者或管理者向管理者发送对五种 Request 报文的响应, 并提供差错码、差错状态等信息
3 (A3)	SetRequest	管理者对代理的一个或多个 MIB 变量的值进行设置
5 (A5)	GetBulkRequest	管理者从代理读取大数据块的值 (如大的列表中的值)
6 (A6)	InformRequest	管理者从另一远程管理者读取该管理者控制的代理中的变量值
7 (A7)	SNMPv2Trap	代理向管理者报告代理中发生的异常事件
8 (A8)	Report	在管理者之间报告某些类型的差错, 目前尚未定义

和大多数 TCP/IP 协议不一样, SNMP 报文没有固定的字段。相反, 它们使用标准 ASN.1 编码。因此, SNMP 报文用人工进行编码和理解时都比较困难。为此, 在图 6-24 中给出了 SNMPv1 的报文格式。可以看出, 一个 SNMP 报文共由四个部分组成, 即版本、首部、安全参数和 SNMP 报文的数据部分。目前最新版本是版本 3。首部包括报文标识 (message identification)、最大报文长度、报文标志 (message flag)。报文标志占 1 字节, 其中的每一位定义安全类型或其他信息。安全参数用来产生报文摘要 (见下一章的 7.3.1 节)。

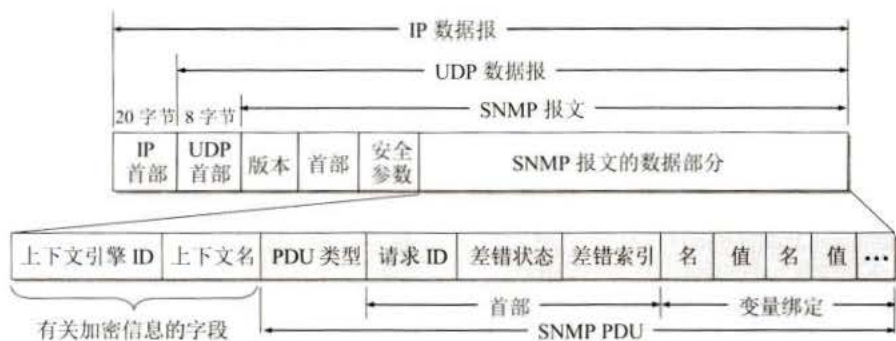


图 6-24 SNMP 的报文格式

从图 6-24 可看出, 在 SNMP PDU 前面还有两个有关加密信息的字段。这是当数据部分需要加密时才使用的两个字段。与网络管理直接相关的是后面的 SNMP PDU 部分。对于表 6-8 给出的前四种 PDU 的格式都是相同的, 即由 PDU 类型、请求 ID、差错状态、差错索引以及变量绑定这几个字段组成。PDU 的各种类型以及类型的编号和 T 字段的编码已在表 6-8 中给出。下面简单介绍一下其他字段的作用。

(1) 请求 ID (request ID) 由管理进程设置的 4 字节整数值。代理进程在发送响应报文时也要返回此请求 ID。由于管理进程可同时向许多代理发出请求读取变量值的报文, 因此设置了请求 ID 可使管理进程能够识别返回的响应是对应于哪一个请求报文。

(2) 差错状态 (error status) 在请求报文中, 这个字段是零。当代理进程响应时, 就填入 0~18 中的一个数字。例如 0 表示 noError (一切正常), 1 表示 tooBig (代理无法把回答装入到一个 SNMP 报文之中), 2 表示 noSuchName (操作指明了一个不存在的变量), 3 表示 badValue (无效值或无效语法), 等等[RFC 3416]。

(3) 差错索引(error index) 在请求报文中, 这个字段是零。当代理进程响应时, 若出现 noSuchName, badValue 或 readOnly 的差错, 代理进程就设置一个整数, 指明有差错的变量在变量列表中的偏移。

(4) 变量绑定(variable-bindings) 指明一个或多个变量的名和对应的值。在请求报文中, 变量的值应忽略(类型是 NULL)。

为了大致了解 ASN.1 给出的定义的形式, 下面举出定义 GetRequest-PDU 的例子。两个连字符“--”后面的是注解。

```
Get-request-PDU ::= [0]                                --[0]表示上下文类, 编号为 0
IMPLICIT SEQUENCE {                                    --类型是 SEQUENCE
    request-id      integer32,                          --变量 request-id 的类型是 integer32
    error-status     INTEGER {0..18},                  --变量 error-status 取值为 0 ~ 18 的整数
    error-index      INTEGER {0..max-bindings},         --变量 error-index 取值为 0 ~ max-bindings 的整数
    variable-bindings VarBindList }                   --变量 variable-binding 的类型是 VarBindList
```

但变量 VarBindList 是什么类型呢? 还需要继续定义(这里从略)。上面 ASN.1 定义中的第二行中的 IMPLICIT 叫作隐式标记, 是为了在进行编码时可省去对 IMPLICIT 后面的类型(SEQUENCE)的编码, 使最后得出的编码更加简洁。

下面我们假定管理者发送 GetRequest-PDU, 为的是从某路由器的代理进程获得“收到 UDP 数据报的数目”的信息。从图 6-21 可以查出, mib-2 下面第 7 个节点是 udp, 而 udp 节点下面的第一个节点就是 udpInDatagrams。由于这个节点已经是叶节点(即没有连接在它下面的子节点了), 读取这个节点的数值时应在节点标识符后面加上 0, 即 1.3.1.1.2.1.7.1.0。这样, 可得出 GetRequest-PDU 的 ASN.1 编码, 如图 6-25 所示。

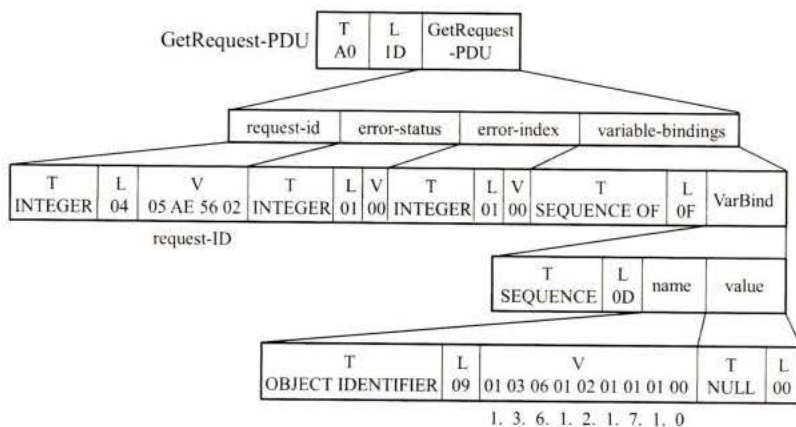


图 6-25 GetRequest-PDU 的 ASN.1 编码

可以把图中各字段的十六进制编码表示如下。

A0 1D	-- GetRequest-PDU, 上下文类型, 长度 1D ₁₆ = 29
02 04 05 AE 56 02	-- INTEGER 类型, 长度 04 ₁₆ , request-id = 05 AE 56 02
02 01 00	-- INTEGER 类型, 长度 01 ₁₆ , error status = 00 ₁₆
02 01 00	-- INTEGER 类型, 长度 01 ₁₆ , error index = 00 ₁₆
30 0F	-- SEQUENCE OF 类型, 长度 0F ₁₆ = 15
30 0D	-- SEQUENCE 类型, 长度 0D ₁₆ = 13

06 09 01 03 06 01 02 01 07 01 00 -- OBJECT IDENTIFIER 类型, 长度 09₁₆, udpInDatagrams
05 00 -- NULL 类型, 长度 00₁₆

6.8 应用进程跨越网络的通信

在这以前我们已经讨论了互联网使用的几种常用的应用层协议, 这些应用协议使广大用户可以更加方便地利用互联网的资源。

现在的问题是: 如果我们还有一些特定的应用需要互联网的支持, 但这些应用又不能直接使用已经标准化的互联网应用协议, 那么我们应当做哪些工作? 要回答这个问题, 实际上就是要了解下面要介绍的系统调用和应用编程接口。这些问题需要一门专门的课程来学习, 我们在这里只能给出一些初步的概念。

6.8.1 系统调用和应用编程接口

大多数操作系统使用系统调用(system call)的机制在应用程序和操作系统之间传递控制权。对程序员来说, 系统调用和一般程序设计中的函数调用非常相似, 只是系统调用是将控制权传递给了操作系统。图 6-26 说明了多个应用进程使用系统调用的机制。

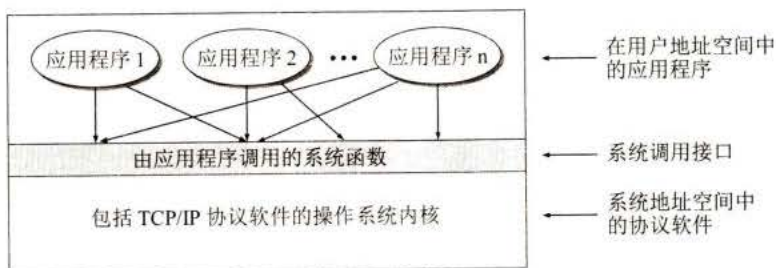


图 6-26 多个应用进程使用系统调用的机制

当某个应用进程启动系统调用时, 控制权就从应用进程传递给了系统调用接口。此接口再把控制权传递给计算机的操作系统。操作系统把这个调用转给某个内部过程, 并执行所请求的操作。内部过程一旦执行完毕, 控制权就又通过系统调用接口返回给应用进程。总之, 只要应用进程需要从操作系统获得服务, 就要把控制权传递给操作系统, 操作系统在执行必要的操作后把控制权返还给应用进程。因此, 系统调用接口实际上就是应用进程的控制权和操作系统的控制权进行转换的一个接口。由于应用程序在使用系统调用之前要编写一些程序, 特别是需要设置系统调用中的许多参数, 因此这种系统调用接口又称为应用编程接口 API (Application Programming Interface)。API 从程序设计的角度定义了许多标准的系统调用函数。应用进程只要使用标准的系统调用函数就可得到操作系统的服务。因此从程序设计的角度看, 也可以把 API 看成是应用程序和操作系统之间的接口。

现在 TCP/IP 协议软件已驻留在操作系统中。由于 TCP/IP 协议族被设计成能运行在多种操作系统的环境中, 因此 TCP/IP 标准没有规定应用程序与 TCP/IP 协议软件如何接口的细节, 而是允许系统设计者能够选择有关 API 的具体实现细节。目前, 只有几种可供应用程序使用 TCP/IP 的应用编程接口 API。这里最著名的就是美国加利福尼亚大学伯克利分校为 Berkeley UNIX 操作系统定义的一种 API, 它又称为套接字接口(socket interface) (或插口接口)。微软公司在其操作系统中采用了套接字接口 API, 形成了一个稍有不同的 API, 并称

之为 Windows Socket, 简称为 WinSock。AT&T 为其 UNIX 系统 V 定义了一种 API, 简称为 TLI (Transport Layer Interface)。

我们知道, 若要让计算机做某件事情, 就要编写使计算机能理解的程序。在网络环境下的计算机应用都有一个共同特点, 这就是: 位于不同地点的计算机要通过网络进行通信。从另一种角度看, 计算机之间的通信就是本计算机要读取另一个地点的计算机中的数据, 或者要把数据从本计算机写入到另一个地点的计算机中。这种“读取”和“写入”的过程都要用到上面所说的系统调用。

在讨论网络编程时常常把套接字作为应用进程和运输层协议之间的接口, 图 6-27 表示这一概念。图中假定了运输层使用 TCP 协议 (若使用 UDP 协议, 情况也是类似的, 只是 UDP 是无连接的。通信的两端仍然可用两个套接字来标志)。现在套接字已成为计算机操作系统内核的一部分。

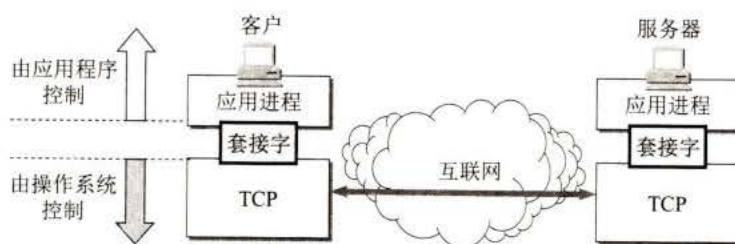


图 6-27 套接字成为应用进程与运输层协议的接口

请注意: 在套接字以上的进程是受应用程序控制的, 而在套接字以下的运输层协议软件则是受计算机操作系统的控制。因此, 只要应用程序使用 TCP/IP 协议进行通信, 它就必须通过套接字与操作系统交互 (这就要使用系统调用函数) 并请求其服务。我们应当注意到, 应用程序的开发者对套接字以上的应用进程具有完全的控制, 但对套接字以下的运输层却只有很少的控制, 例如, 可以选择运输层协议 (TCP 或 UDP) 以及一些运输层的参数 (如最大缓存空间和最大报文长度等)。

当应用进程 (客户或服务器) 需要使用网络进行通信时, 必须首先发出 socket 系统调用, 请求操作系统为其创建一个“套接字”。这个调用的实际效果是请求操作系统把网络通信所需要的一些系统资源 (存储器空间、CPU 时间、网络带宽等) 分配给该应用进程。操作系统为这些资源的总和用一个叫作套接字描述符 (socket descriptor) 的号码 (小的整数) 来表示, 然后把这个套接字描述符返回给应用进程。此后, 应用进程所进行的网络操作 (建立连接、收发数据、调整网络通信参数等) 都必须使用这个套接字描述符。所以, 几乎所有的网络系统调用都把这个套接字描述符作为套接字的许多参数中的第一个参数。在处理系统调用的时候, 通过套接字描述符, 操作系统就可以识别出应该使用哪些资源来完成应用进程所请求的服务。通信完毕后, 应用进程通过一个关闭套接字的 close 系统调用通知操作系统回收与该套接字描述符相关的所有资源。由此可见, 套接字是应用进程为了获得网络通信服务而与操作系统进行交互时使用的一种机制。

图 6-28 给出了当应用进程发出 socket 系统调用时, 操作系统所创建的套接字描述符与套接字数据结构的关系。由于在一个进程中可能同时出现多个套接字, 因此需要有一个存放套接字描述符的表, 而每一个套接字描述符有一个指针指向存放套接字的地址。在套接字的

数据结构中有许多参数要填写。图 6-28 中已填写好的参数是协议族 (PF_INET, 表示使用 Internet 的 TCP/IP 协议族) 和服务 (SOCK_STREAM, 表示使用流式服务, 也就是使用 TCP 服务)。在刚刚创建一个新的套接字时, 有灰色背景的四个项目 (本地和远地 IP 地址, 本地和远地端口) 都是未填写的, 因此它和任何机器中的应用进程暂时都还没有联系。

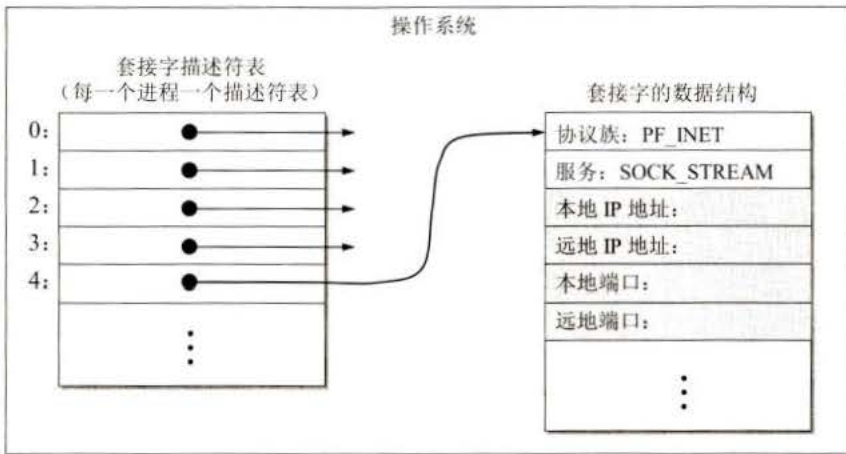


图 6-28 调用 socket 创建套接字

这里要特别强调一下, 在第 5 章 5.3.2 节的最后, 我们曾指出, 同一个名词 socket 可表示多种不同的意思。在本节讨论 socket 系统调用时, 套接字 socket 已不仅仅是 RFC 793 所定义的如公式(5-1)所示的那样, 而是如图 6-28 右边所示的套接字的数据结构。

6.8.2 几种常用的系统调用

下面我们以使用 TCP 的服务为例介绍几种常用的系统调用。

1. 连接建立阶段

当套接字被创建后, 它的端口号和 IP 地址都是空的, 因此应用进程要调用 bind (绑定) 来指明套接字的本地地址 (本地端口号和本地 IP 地址)。在服务器端调用 bind 时就是把熟知端口号和本地 IP 地址填写到已创建的套接字中。这就叫作把本地地址绑定到套接字。在客户端也可以不调用 bind, 这时由操作系统内核自动分配一个动态端口号 (通信结束后由系统收回)。

服务器在调用 bind 后, 还必须调用 listen (收听) 把套接字设置为被动方式, 以便随时接受客户的服务请求。UDP 服务器由于只提供无连接服务, 不使用 listen 系统调用。

服务器紧接着就调用 accept (接受), 以便把远地客户进程发来的连接请求提取出来。系统调用 accept 的一个变量就是要指明是从哪一个套接字发起的连接。

调用 accept 要完成的动作较多。这是因为一个服务器必须能够同时处理多个连接。这样的服务器常称为并发方式 (concurrent) 工作的服务器。可以有多种方法实现这种并发方式。图 6-29 所示的是一种实现方法。

主服务器进程 M (就是通常所说的服务器进程) 一调用 accept, 就为每一个新的连接请求创建一个新的套接字, 并把这个新创建的套接字的标识符返回给发起连接的客户方。与此同时, 主服务器进程还要创建一个从属服务器进程 (如图 6-29 中的 S₁) 来处理新建立的连接。这样, 从属服务器进程用这个新创建的套接字和客户进程建立连接, 而主服务器进

程用原来的套接字重新调用 `accept`，继续接受下一个连接请求。在已建立的连接上，从属服务器进程就使用这个新创建的套接字传送和接收数据。数据通信结束后，从属服务器进程就关闭这个新创建的套接字，同时这个从属服务器也被撤销。

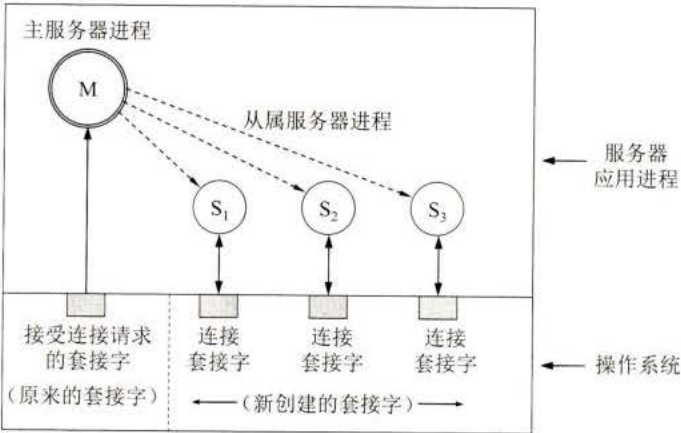


图 6-29 并发方式工作的服务器

总之，在任一时刻，服务器中总是有一个主服务器进程和零个或多个从属服务器进程。主服务器进程用原来的套接字接受连接请求，而从属服务器进程用新创建的套接字（在图 6-29 中注明是“连接套接字”）和相应的客户建立连接并可进行双向传送数据。

以上介绍的是服务器为了接受客户端发起的连接请求而进行的一些系统调用。现在看一下客户端的情况。当使用 TCP 协议的客户已经调用 `socket` 创建了套接字后，客户进程就调用 `connect`，以便和远地服务器建立连接（这就是**主动打开**，相当于客户发出的连接请求）。在 `connect` 系统调用中，客户必须指明远地端点（即远地服务器的 IP 地址和端口号）。

2. 数据传送阶段

客户和服务器都在 TCP 连接上使用 `send` 系统调用传送数据，使用 `recv` 系统调用接收数据。通常客户使用 `send` 发送请求，而服务器使用 `send` 发送回答。服务器使用 `recv` 接收客户用 `send` 调用发送的请求。客户在发完请求后用 `recv` 接收回答。

调用 `send` 需要三个变量：数据要发往的套接字的描述符、要发送的数据的地址以及数据的长度。通常 `send` 调用把数据复制到操作系统内核的缓存中。若系统的缓存已满，`send` 就暂时阻塞，直到缓存有空间存放新的数据。

调用 `recv` 也需要三个变量：要使用的套接字的描述符、缓存的地址以及缓存空间的长度。

3. 连接释放阶段

一旦客户或服务器结束使用套接字，就把套接字撤销。这时就调用 `close` 释放连接和撤销套接字。

图 6-30 画出了上述的一些系统调用的使用顺序。有些系统调用在一个 TCP 连接中可能会循环使用。

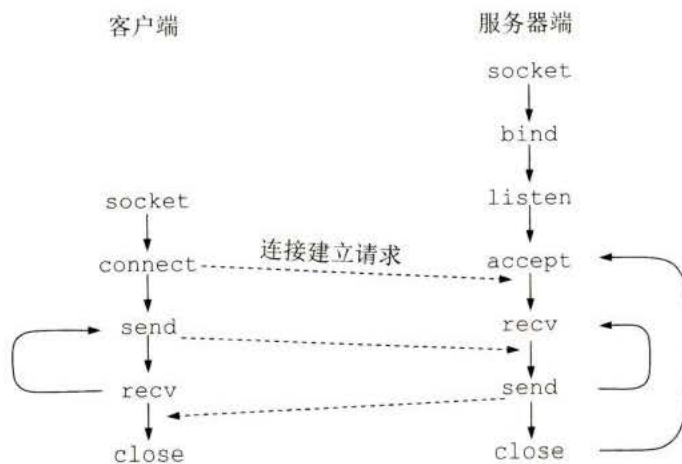


图 6-30 系统调用使用顺序的例子

UDP 服务器由于只提供无连接服务，因此不使用 `listen` 和 `accept` 系统调用。

6.9 P2P 应用

我们在第 1 章的 1.3.1 节中已经简单地介绍了 P2P 应用的概念。现在我们将进一步讨论 P2P 应用的若干工作原理。

P2P 应用就是指具有 P2P 体系结构的网络应用。所谓 P2P 体系结构就是在这样的网络应用中，没有（或只有极少数的）固定的服务器，而绝大多数的交互都是使用对等方式（P2P 方式）进行的。

P2P 应用的范围很广，例如，文件分发、实时音频或视频会议、数据库系统、网络服务支持（如 P2P 打车软件、P2P 理财等）。限于篇幅，下面只介绍最常用的 P2P 文件分发的工作原理。

P2P 文件分发不需要使用集中式的媒体服务器，而所有的音频/视频文件都是在普通的互联网用户之间传输的。这其实是相当于有很多（有时达到上百万个）分散在各地的媒体服务器（由普通用户的计算机充当这种媒体服务器）向其他用户提供所要下载的音频/视频文件。这种 P2P 文件分发方式解决了集中式媒体服务器可能出现的瓶颈问题。

目前在互联网流量中，P2P 工作方式下的文件分发已占据了最大的份额，比万维网应用所占的比例大得多。因此单纯从流量的角度看，P2P 文件分发应当是互联网上最重要的应用。现在 P2P 文件分发不仅传送音频文件 MP3，而且还传送视频文件（10 ~ 1000 MB，或更大）、各种软件和图像文件。

6.9.1 具有集中目录服务器的 P2P 工作方式

最早使用 P2P 工作方式的是 Napster。这个名称来自 1999 年美国东北大学的新生 Shawn Fanning 所写的一个叫作 Napster 的软件。利用这个软件就可通过互联网免费下载各种 MP3

音乐。Napster 的出现使 MP3 成为网络音乐事实上的标准^①。

Napster 能够搜索音乐文件，能够提供检索功能。所有音乐文件的索引信息都集中存放在 Napster 目录服务器中。这个目录服务器起着索引的作用。使用者只要查找目录服务器，就可知道应从何处下载所要的 MP3 文件。在 2000 年，Napster 成为互联网上最流行的 P2P 应用，并占据互联网上的通信量中相当大的比例。

这里的关键就是运行 Napster 的所有用户，都必须及时向 Napster 的目录服务器报告自己已经存有哪些音乐文件。Napster 目录服务器就用这些用户信息建立起一个动态数据库，集中存储了所有用户的音乐文件信息（即对象名和相应的 IP 地址）。当某个用户想下载某个 MP3 文件时，就向目录服务器发出查询（这个过程仍是传统的客户-服务器方式），目录服务器检索出结果后向用户返回存放这一文件的计算机 IP 地址，于是这个用户就可以从中选取一个地址下载想要得到的 MP3 文件（这个下载过程就是 P2P 方式）。可以看出，Napster 的文件传输是分散的（P2P 方式），但文件的定位则是集中的（客户-服务器方式）。

图 6-31 是 Napster 的工作过程的示意图。假定 Napster 目录服务器已经建立了其用户的动态数据库。图中给出了某个用户要下载音乐文件的主要交互过程。

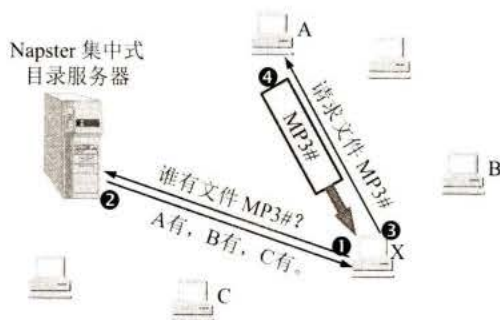


图 6-31 Napster 的工作过程

- ① 用户 X 向 Napster 目录服务器查询（客户-服务器方式）谁有文件 MP3#。
- ② Napster 目录服务器回答 X：有三个地点有文件 MP3#，即 A、B 和 C（给出了这三个地点的 IP 地址）。于是用户 X 得知所需的文件 MP3# 的三个下载地点。
- ③ 用户 X 可以随机地选择三个地点中的任一个，也可以使用 PING 报文寻找最方便下载的一个。在图 6-31 中，我们假定 X 向 A 发送下载文件 MP3# 的请求报文。现在 X 和 A 都使用 P2P 方式通信，互相成为对等方，X 是临时的客户，而对等方 A 是临时的服务器。
- ④ 对等方 A（现在作为服务器）把文件 MP3# 发送给 X。

这种集中式目录服务器的最大缺点就是可靠性差，而且会成为其性能的瓶颈（尤其是在用户数非常多的情况下）。更为严重的是这种做法侵犯了唱片公司的版权。虽然 Napster 网站并没有直接非法复制任何 MP3 文件（Napster 网站不存储任何 MP3 文件，因而并没有

① 注：通常 CD 光盘中的音乐都采用 wav 格式，即采用标准的 PCM 编码。MP3 则进行了压缩，把一些人耳朵不太能够听出来的部分频率分量压缩掉了。压缩比一般在 1:4 到 1:12 之间。例如，压缩比为 1:10 就表明 10MB 的 wav 格式音乐，可压缩成 1 MB 的 MP3 格式，而音质的下降并不太明显。

直接侵犯版权)，但法院还是判决 Napster 属于“间接侵害版权”，因此在 2000 年 7 月底 Napster 网站就被迫关闭了。

6.9.2 具有全分布式结构的 P2P 文件共享程序

在第一代 P2P 文件共享网站 Napster 关闭后，开始出现了以 Gnutella 为代表的第二代 P2P 文件共享程序。Gnutella 是一种采用全分布方法定位内容的 P2P 文件共享应用程序。Gnutella 与 Napster 最大的区别就是不使用集中式的目录服务器进行查询，而是使用洪泛法在大量 Gnutella 用户之间进行查询。为了不使查询的通信量过大，Gnutella 设计了一种**有限范围的洪泛查询**。这样可以减少倾注到互联网的查询流量，但由于查询的范围受限，因而这也影响到查询定位的准确性。

为了更加有效地在大量用户之间使用 P2P 技术下载共享文件，最近几年已经开发出很多种第三代 P2P 共享文件程序[KURO17]，它们使用分散定位和分散传输技术。如 KaZaA，电骡 eMule，比特洪流 BT (Bit Torrent)等。

下面对比特洪流 BT 的主要特点进行简单的介绍。

在 P2P 的文件分发应用中，2001 年由 Brahm Cohen 开发的 BitTorrent（中文意思是“比特洪流”）是很具代表性的一个。取这个名称的原因就是 BitTorrent 把参与某个文件分发的所有对等方的集合称为一个**洪流(torrent)**。为了方便，下面我们使用 BitTorrent 的简称 BT。BT 把对等方下载文件的数据单元称为**文件块(chunk)**，一个文件块的长度是固定不变的，例如，典型的数值是 256 KB。当一个新的对等方加入某个洪流时，一开始它并没有文件块。但新的对等方逐渐地能够下载到一些文件块。而与此同时，它也为别的对等方上传一些文件块。某个对等方获得了整个的文件后，可以立即退出这个洪流（相当于自私的用户），也可继续留在这个洪流中，为其他的对等方上传文件块（相当于无私的用户）。加入或退出某个洪流可在任何时间完成（即使在某个文件还没有下载完毕时），也是完全自由的。

BT 的协议相当复杂[W-BT]。下面讨论其基本机制。

每一个洪流都有一个基础设施节点，叫作**追踪器(tracker)**。当一个对等方加入洪流时，必须向追踪器**登记**（或称为**注册**），并周期性地通知追踪器它仍在洪流中。追踪器因而就跟踪了洪流中的对等方。一个洪流中可以拥有少到几个多到几百或几千个对等方。

我们用图 6-32 来进一步说明 BT 的工作原理。当一个新的对等方 A 加入洪流时，追踪器就随机地从参与的对等方集合中选择若干个（例如，30 个），并把这些对等方的 IP 地址告诉 A。于是 A 就和这些对等方建立了 TCP 连接。我们称所有与 A 建立了 TCP 连接的对等方为“**相邻对等方**”(neighboring peers)。在图 6-32 中我们在上面的**覆盖网络**中画出了 A 有三个相邻对等方 (B, C 和 D)。这些相邻对等方的数目是动态变化的，有的对等方可能不久就离开了，但不久后又又有新加入进来的对等方。请注意，实际的网络拓扑可能是非常复杂的（参见图 6-32 覆盖网络下面的实际网络图）。我们知道，TCP 连接只是个逻辑连接，而每一个 TCP 连接可能会穿越很多的网络。因此我们在讨论问题时，可以利用实际网络上面的一个更加简洁的覆盖网络，这个覆盖网络忽略了实际网络的许多细节，使问题的讨论更加方便。在覆盖网络中，A 的三个相邻对等方就看得很清楚。

在任何时刻，每一个对等方可能只拥有某文件的一个文件块子集，而不同的对等方所拥有的文件块子集也不会完全相同。对等方 A 将通过 TCP 连接周期性地向其相邻对等方索取它们拥有的文件块列表。根据收到的文件块列表，A 就知道了应当请求哪一个相邻对等方

把哪些自己缺少的文件块发送过来。

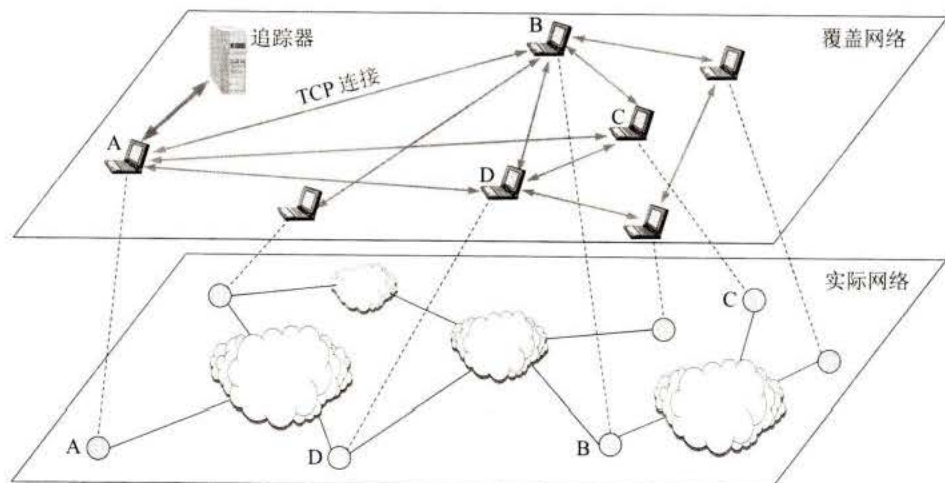


图 6-32 在覆盖网络中对等方的相邻关系的示意图

图 6-33 是对等方之间互相传送数据块的示意图。例如，A 向 B、C 和 D 索取数据块，但 B 同时也向 C 和 D 传送数据块，D 和 C 还互相传送数据块。由于 P2P 对等用户的数量非常多，因此，从不同的对等方获得不同的数据块，然后组装成整个的文件，一般要比仅从一个地方下载整个的文件要快很多。

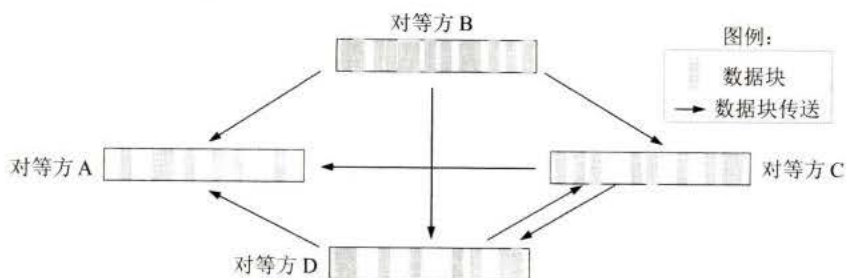


图 6-33 对等方之间互相传送文件数据块

然而 A 必须做出两个重要决定。第一，哪些文件块是首先需要向其相邻对等方请求的？第二，在很多向 A 请求文件块的相邻对等方中，A 应当向哪些相邻对等方发送所请求的文件块？

对于第一个问题，A 要使用叫作**最稀有的优先(rarest first)**的技术。我们知道，凡是 A 所缺少的而正好相邻对等方已拥有的文件块，都应当去索取。可能其中的某些文件块，很多相邻对等方都有（即文件块的副本很多），这就是“不稀有的”文件块，以后可慢慢请求。如果 A 所缺少的文件块在相邻对等方中的副本很少，那就是“很稀有的”。因此，A 首先应当请求副本最少的文件块（即最稀有的）。否则，一旦拥有最稀有文件块的对等方退出了洪流，就会影响 A 对所缺文件块的收集。

对于第二个问题，BT 采用了一种更加机灵的算法，其基本思想就是：凡当前有以最高数据率向 A 传送文件块的某相邻对等方，A 就优先把所请求的文件块传送给该相邻对等方。具体来说，A 持续地测量从其相邻对等方接收数据的速率，并确定速率最高的 4 个相邻对等方。接着，A 就把文件块发送给这 4 个相邻对等方。每隔 10 秒钟，A 还要重新计算数据率，

然后可能修改这 4 个对等方。在 BT 的术语中, 这 4 个对等方叫作已疏通的或无阻碍的 (unchoked) 对等方。更重要的是, 每隔 30 秒, A 要随机地找一个另外的相邻对等方 B, 并向其发送文件块。这样, A 有可能成为 B 的前 4 位上传文件块的提供者。在此情况下, B 也有可能向 A 发送文件块。如果 B 发送文件块的速率足够快, 那么 B 也有可能进入 A 的前 4 位上传文件块的提供者。这样做的结果是, 这些对等方相互之间都能够以令人满意的速率交换文件块。

6.9.3 P2P 文件分发的分析

我们从一个例子开始, 来讨论 P2P 文件分发中的几个重要概念[KURO17]。

在图 6-34 中, 有 N 台主机要从互联网上的服务器下载一个大文件, 其长度为 F bit。在图中我们把这个文件也记为 F 。按照习惯, 从互联网传送数据到主机, 叫作下载(download), 而反过来传送数据, 即从主机向互联网传送, 则称为上传(upload)或上载。服务器的文件是供互联网上的用户享用的, 因此服务器的文件只是单方向上传到互联网。我们把服务器的上传速率记为 u_s , 单位是 bit/s。再假定主机与互联网连接的链路的上传速率和下载速率分别为 u_i 和 d_i , 单位都是 bit/s。我们还假定互联网的核心部分不会产生拥塞。瓶颈只会发生在服务器的接入链路, 或者是某些主机的接入链路。

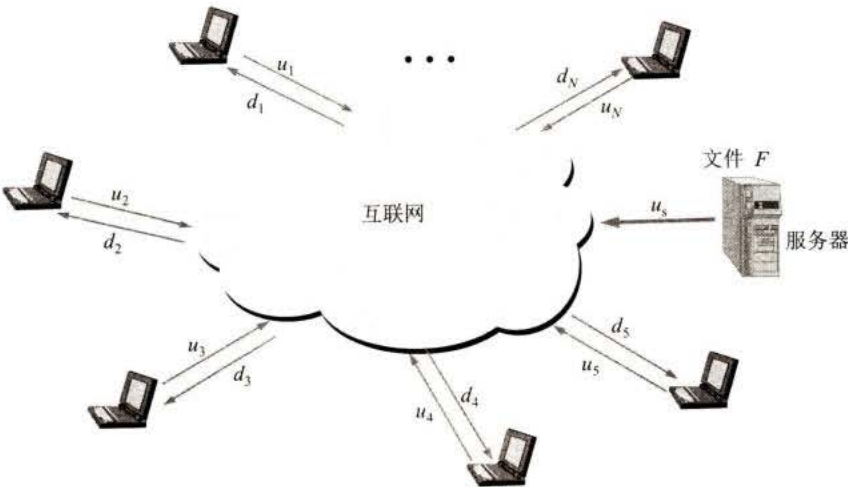


图 6-34 文件分发的例子

我们先在传统的客户-服务器方式下, 计算给所有主机分发完毕的最短时间 T_{cs} 。

从服务器端考虑, N 台主机共需要从服务器得到的数据总量 (比特数) 是 NF 。如果服务器能够不停地以其上传速率 u_s 向各主机传送数据, 一直到各主机都收到文件 F , 就需要时间 NF/u_s , 单位是秒。由此可见, T_{cs} 不可能小于 NF/u_s 。

如果 N 台主机都以各自的下载速率不停地下载文件 F , 那么下载速率最慢的主机 (设其下载速率为 d_{min}) 的下载文件时间 (F/d_{min}), 将是 N 个下载时间中最大的一个。由此可见, T_{cs} 也不可能小于 F/d_{min} 。

如果 $NF/u_s \geq F/d_{min}$, 则瓶颈在服务器端的接入链路。这时 $T_{cs} = NF/u_s$ 。

如果 $F/d_{min} \geq NF/u_s$, 则瓶颈在下载最慢的主机的接入链路。这时 $T_{cs} = F/d_{min}$ 。

由此可得出所有主机都下载完文件 F 的最少时间是

$$T_{cs} = \max \left\{ \frac{NF}{u_s}, \frac{F}{d_{\min}} \right\} \quad (6-2)$$

从以上分析可以看出, 若公式(6-2)括号中的第一项大于第二项, 则 T_{cs} 就与主机数 N 成正比。如果主机数增大 1000 倍, 那么文件的分发时间也要增大 1000 倍。

下面讨论在 P2P 方式下, 文件全部分发完毕的最少时间 T_{P2P} 。然而在 P2P 方式下, 文件分发所需的时间较难计算, 这是因为每一台主机在接收文件的同时, 还利用自己的上传能力向其他主机传送文件。文件传送所需的时间取决于主机向对等方传送文件的具体方式。但是, 我们还是可以导出文件分发所需的最少时间的表达式。

在文件分发开始时, 只有服务器有文件 F 。服务器必须把文件 F 的每一个比特通过接入链路传送到互联网 (至少要传送一次)。因此文件分发的最少时间不可能小于 F/u_s 。和客户-服务器方式相比, 在 P2P 方式下, 服务器不需要一遍一遍地发送文件 F , 因为互联网上的其他主机 (即对等方) 可以代替服务器向其他对等方分发文件 F 。

在 P2P 方式下, 下载速率最慢的主机 (设其下载速率为 $d_{\min}$) 下载文件 F 的时间是 $F/d_{\min}$, 这是 N 个对等方下载时间中最大的一个。可见文件分发的最少时间不可能小于 $F/d_{\min}$ 。这个结论和客户-服务器方式是一样的。

整个系统中所有主机 (包括服务器) 的上传速率之和是 $u_T = u_s + u_1 + u_2 + \dots + u_N$ 。因此, 文件分发的最少时间也不可能小于 NF/u_T 。

这样, 我们得出在 P2P 方式下所有主机都下载完文件 F 的最少时间的下限是

$$T_{P2P} \geq \max \left\{ \frac{F}{u_s}, \frac{F}{d_{\min}}, \frac{NF}{u_T} \right\} \quad (6-3)$$

在公式(6-3)的推导过程中, 我们假定每一个对等方只要收到一个比特就立即上传到互联网的其他对等方。但实际上是把收到的若干个比特组成一个数据块后再上传出去。但是当文件 F 很大时, 我们也可以在公式(6-3)中取等号, 作为文件 F 的最少分发时间 T_{P2P} 的近似值。

有一种情况最值得我们注意。这就是对等方的数目 N 非常大, 因此在公式(6-3)的括号中的最后一项的值将大于前两项的值。这样, T_{P2P} 值的下限就近似为 NF/u_T 。

我们再假定一些数据。设所有的对等方的上传速率都是 u , 并且 $F/u = 1$ 小时, 所有对等方的下载速率都不小于服务器的上传速率, 因而不会对我们的计算产生影响。我们再设服务器的上传速率 $u_s = 10u$ 。当 $N = 30$ 时, 用公式(6-3)算出所有主机都下载完文件 F 的最少时间的下限是 $T_{P2P} = 0.75 F/u = 0.75$ 小时 (这个数值一定小于 1 小时)。如果采用客户-服务器方式, 则当 $N = 30$ 时, 所有主机都下载完文件 F 的最少时间是 $T_{cs} = NF/u_s = 3$ 小时。

6.9.4 在 P2P 对等方中搜索对象

在 P2P 文件系统中, 对等方用户的数量非常多, 并且处于一种无序的状态。任何一个对等方可以随时加入进来或随时退出。在这种情况下, 怎样有效地找到所需的文件, 也就是怎样有效地定位对等方及其资源, 乃是 P2P 系统中的一个十分重要的问题。

限于篇幅, 我们在这里只简单介绍一下怎样利用散列函数来定位对等方。

我们知道, Gnutella 是一种采用全分布方法定位内容的 P2P 文件共享应用程序, 它解决了集中式目录服务器所造成的瓶颈问题。然而 Gnutella 是在非结构化的覆盖网络中采用查询洪泛的方法来进行查找的, 因此查找的效率较低。现在比较好的查找方法是设法构建一种分布式数据库, 以进行对等方及其资源的定位。这种分布式数据库在概念上并不复杂, 只要能够支持大量对等方 (可能有几百万个) 进行索引查找即可。存储在数据库中的信息只有两个部分:

(1) 要查找的资源名 K (例如, 电影或歌曲的名字)。资源名也可称为关键字。

(2) 存放该对象的节点的 IP 地址 N 。有的 IP 地址还附带有端口号。

存放在数据库中的信息就是大量成对出现的 (资源名 K , 节点的 IP 地址 N)。在查找某资源名 K 时, 只要在数据库中查找到匹配的资源名 K , 数据库就能够返回对应的节点的 IP 地址 N 。所以问题的关键就是要设法把每个资源名 K 存放在一个非常便于查找的地方。

细心的读者可能会联想到曾在前面 6.1 节讨论的 DNS 域名系统。DNS 是根据主机的域名来查找其 IP 地址, 这和 P2P 的情况有相似之处。但我们知道, 主机的域名是结构化的命名系统, 因此域名服务器可以划分为几种不同的级别 (如根服务器等) 便于查找。但 P2P 系统则不同, 其资源名是非结构化的。因此不能套用 DNS 的那种查找方法。

前面已经讲过, Napster 在一个集中式目录服务器中构建的查找数据库虽然很简单, 但性能上却有瓶颈。在 P2P 系统中, 应怎样构建分布式的 P2P 数据库? 让每个对等方都拥有所有对等方 IP 地址的列表是不可行的。让所有成对出现的 (资源名 K , IP 地址 N) 随机地分散到各对等方也是不可行的。因为这将使查找对象的次数过大, 无法使用。现在广泛使用的索引和查找技术叫作分布式散列表 DHT (Distributed Hash Table)。DHT 也可译为分布式哈希表, 它是由大量对等方共同维护的散列表。基于 DHT 的具体算法已有不少, 如 Chord, Pastry, CAN (Content Addressable Network), 以及 Kademlia 等。下面简单介绍广泛使用的 Chord 算法, 这是美国麻省理工大学于 2001 年提出的[STOI01]。

分布式散列表 DHT 利用散列函数, 把资源名 K 及其存放的节点 IP 地址 N 都分别映射为资源名标识符 KID 和节点标识符 NID。如果所有的对等方都使用散列函数 SHA-1 (我们在下一章 7.3.1 节还要介绍 SHA-1 在网络安全方面的应用), 那么通过散列得出的标识符 KID 和 NID 都是 160 位二进制数字, 且其数值范围在 $[0, 2^{160} - 1]$ 之间。虽然从理论上讲, 散列函数 SHA-1 是多对一的函数, 但实际上不同输入得到相同的输出的概率是极小的。此外, 通过 SHA-1 映射得到的标识符能够比较均匀而稀疏地分布在 Chord 环上。为便于讨论, 我们假定现在标识符只有 5 位二进制数字, 也就是说, 所有经散列函数得出的标识符的数值范围都在 $[0, 31]$ 之间。Chord 把节点按标识符数值从小到大沿顺时针排列成一个环形覆盖网络 (见图 6-35(a)), 并按照下面的规则进行映射:

(1) 节点标识符 NID 按照其标识符值映射到 Chord 环上对应的点, 见图 6-35(a)中标有 NID 的小圆点, 如 N4, N7, N10, N20, N26 和 N30。

(2) 资源名标识符 KID 则按照其标识符值映射到与其值最接近的下一个 NID, 见图 6-36(a)中标有 KID 的小方块。所谓“最接近的下一个”NID 就是指: 从 KID 值开始, 按顺时针方向沿 Chord 环遇到的下一个 NID。例如, K31 和 K2 应放在 N4, 因为在环上从 31 和 2 按顺时针方向遇到的下一个 NID 是 N4。同理, K8, K12, K23 和 K29 应分别放在 N10, N20, N26 和 N30。如果碰巧同时出现 K29 和 N29 (这种概率极小), 那么 K29 就应当放在 N29。

请注意: 在图 6-35 中, K31 和 K2 都放在 N4, 这表示要查找存放资源 K31 或 K2 的节

点的 IP 地址, 就应当到节点 N4 去查找。请注意, 资源 K31 和 K2 并非存放在节点 N4。

这就是说, 每个资源由 Chord 环上与其标识符值最接近的下一个节点提供服务。我们再强调一下, Chord 环并非实际的网络。在 Chord 环上相邻的节点, 在地理上很可能相距非常远。

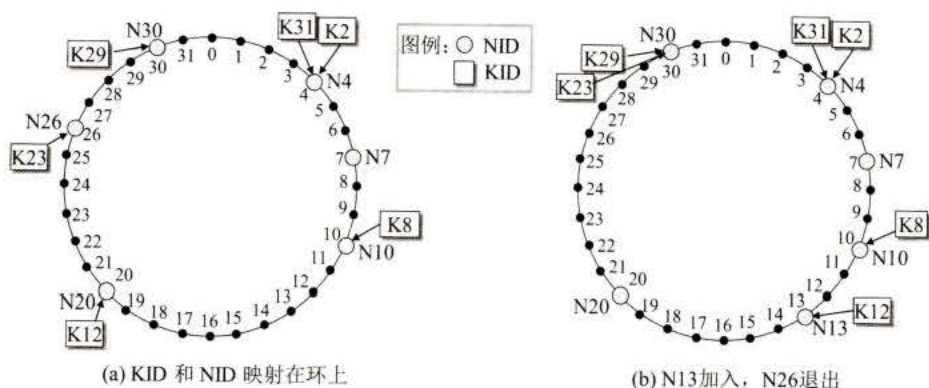


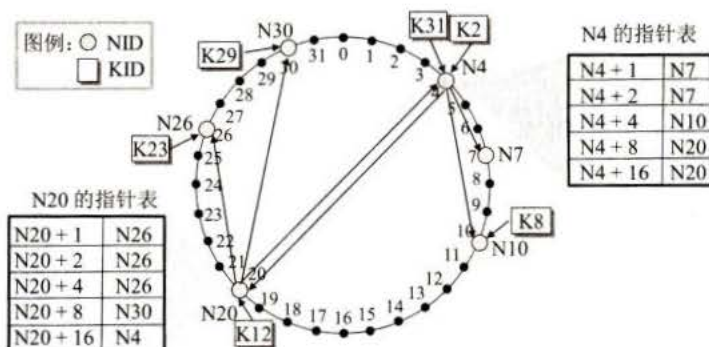
图 6-35 基于 DHT 的 Chord 环

Chord 环上的每一个节点都要维护两个指针变量, 一个指向其后继节点, 而另一个指向其前任节点。例如, 在图 6-35(a)中, N10 的后继节点是 N20 (N10 沿顺时针方向的下一个节点), 其前任节点是 N7 (N10 沿逆时针方向的前一个节点)。如果一个新的节点 N13 加入进来, 那么 N20 的前任节点就变为 N13, 因而 K12 就要从 N20 的位置移到 N13, 同时 N10 的后继节点就变为 N13 (见图 6-35(b))。此外, 如果节点 N26 退出, 那么 K23 就要移到 N30, 而 N30 的前任节点就变为 N20, 同时 N20 的后继节点变为 N30。

在这样的 Chord 环上查找资源, 从理论上讲, 任何一个节点, 只要从其后继节点一个个地遍历查找下去, 一定可以找到所查询的资源。可见要定位一个资源, 平均需要沿环发送查找报文 $N/2$ 个, 或遍历 $O(N)$ 个节点 (N 为环上的总节点数)。显然, 这种顺序查找的方法效率很低。

为了加速查找, 在 Chord 环上可以增加一些指针表(finger table), 它又称为路由表或查找器表。若 Chord 环上的标识符有 m 位 (现在 $m = 5$), 则在节点 n 上的指针表可设置不超过 m 个指针, 指向其后继的节点。我们先看图 6-36 中节点 N4 的指针表。指针表中的第 2 列是从 N4 可以指向的多个后继节点。本来每一个节点仅仅指向沿顺时针方向的下一个后继节点, 但现在则指向多个后继节点 (在本例中就是 N7, N10 和 N20)。第 1 列的第 i 行是计算 $(N4 + 2^{i-1})$, 用来得出后继节点。例如, 第 4 行 $i = 4$, 算出 $(N4 + 2^{i-1}) = N4 + 8 = 12$, 而 Chord 环上的节点 12 的后继节点是 N20。图中还画出了从 N4 到这几个后继节点的连线 (这些连线就是 Chord 环上的弦, Chord 名字由此得出)。还有一点要注意的是, 在 N20 的指针表中的第 5 行, $N20 + 16 = 36$, 但按照模 2^5 运算, $36 \bmod 2^5 = 4$, 恰好节点 4 的后继节点是 N4。

假定在图 6-36 中的节点 N4 要查找 K29。如果用遍历各节点的方法, 则要查找 5 次, 即 $N7 \rightarrow N10 \rightarrow N20 \rightarrow N26 \rightarrow N30$ 。但若利用指针表, 则 N4 首先在自己的指针表中寻找在不到 29 且最接近 29 的节点, 即 N20, 然后把定位资源 K29 的请求发送给 N20。在 N20 的指针表中继续类似的寻找。结果是: 最接近 29 的节点是 N30。这就是存放资源 K29 的节点。这种查找方法类似于二分查找, 只用了两次查找, 定位一个资源仅需 $O(\log_2 N)$ 步。



在 P2P 网络中, 对等方可能相当频繁地加入或退出系统, 这就需要很好地维护这个分布式数据库 (维护各节点的指针和指针表), 而这种维护的工作量可能会很大。当对等方数量非常大时, 究竟采用何种查询机制更加合理, 则需要根据具体情况来确定。

P2P 技术还在不断地改进,但随着 P2P 文件共享程序日益广泛地使用,也产生了一系列的问题有待于解决。这些问题已迫使人们要重新思考下一代互联网应如何演进。例如,音频/视频文件的知识产权就是其中的一个问题。又如,当非法盗版的、或不健康的音频/视频文件在互联网上利用 P2P 文件共享程序广泛传播时,要对 P2P 的流量进行有效的管理,在技术上还是有相当的难度。由于现在 P2P 文件共享程序的大量使用,已经消耗了互联网主干网上大部分的带宽。因此,怎样制定出合理的收费标准,既能够让广大网民接受,又能使网络运营商赢利并继续加大投入,也是目前迫切需要解决的问题。

本章的重要概念

- 本章所讨论的应用程序是为了解决互联网上的某一类应用问题的软件，而这些应用问题的解决又必须通过位于不同主机中的多个应用进程之间的通信和协同工作来完成。应用层协议规定了应用进程在通信时所遵循的各种规则。应用层协议是为应用程序的实现服务的。
- 应用层的许多协议都是基于客户服务器方式的。客户是服务请求方，服务器是服务提供方。
- 域名系统 DNS 是互联网使用的命名系统，用来把便于人们使用的机器名字转换为 IP 地址。DNS 是一个联机分布式数据库系统，并采用客户服务器方式。
- 域名到 IP 地址的解析是由分布在互联网上的许多域名服务器程序（即域名服务器）共同完成的。
- 互联网采用层次树状结构的命名方法，任何一台连接在互联网上的主机或路由器，都有一个唯一的层次结构的名字，即域名。域名中的点和点分十进制 IP 地址中的点没有对应关系。
- 域名服务器分为根域名服务器、顶级域名服务器、权限域名服务器和本地域名服务器。
- 文件传送协议 FTP 使用 TCP 可靠的运输服务。FTP 使用客户服务器方式。一个 FTP 服务器进程可同时为多个客户进程提供服务。在进行文件传输时，FTP 的客户和服务器之间要建立两个并行的 TCP 连接：控制连接和数据连接。实际用于传输文件

的是数据连接。

- 万维网 WWW 是一个大规模的、联机式的信息储藏所，可以非常方便地从互联网上的一个站点链接到另一个站点。
- 万维网的客户程序向互联网中的服务器程序发出请求，服务器程序向客户程序送回客户所要的万维网文档。在客户程序主窗口上显示出的万维网文档称为页面。
- 万维网使用统一资源定位符 URL 来标志万维网上的各种文档，并使每一个文档在整个互联网的范围内具有唯一的标识符 URL。
- 万维网客户程序与服务器程序之间进行交互所使用的协议是超文本传送协议 HTTP。HTTP 使用 TCP 连接进行可靠的传送。但 HTTP 协议本身是无连接、无状态的。HTTP/1.1 协议使用了持续连接（分为非流水线方式和流水线方式）。
- HTTP/2 可使用同一个 TCP 连接把服务器发回的响应并行发回；允许客户复用 TCP 连接进行多个请求；把所有的报文划分为许多较小的二进制编码的帧，采用新的压缩算法，不发送重复的首部字段，大大减小了首部的开销，提高了传输效率。
- 万维网使用超文本标记语言 HTML 来显示各种万维网页面。
- 万维网静态文档是指在文档创作完毕后就存放在万维网服务器中，在被用户浏览的过程中，内容不会改变。动态文档是指文档的内容是在浏览器访问万维网服务器时才由应用程序动态创建的。
- 活动文档技术可以使浏览器屏幕连续更新。活动文档程序可与用户直接交互，并可连续地改变屏幕的显示。
- 在万维网中用来进行搜索的工具叫作搜索引擎。搜索引擎大体上可划分为全文检索搜索引擎和分类目录搜索引擎两大类。
- 电子邮件是互联网上使用最多的和最受用户欢迎的一种应用。电子邮件把邮件发送到收件人使用的邮件服务器，并放在其中的收件人邮箱中，收件人可随时上网到自己使用的邮件服务器进行读取，相当于“电子信箱”。
- 一个电子邮件系统有三个主要组成构件，即：用户代理、邮件服务器和邮件协议（包括邮件发送协议，如 SMTP，和邮件读取协议，如 POP3 和 IMAP）。用户代理和邮件服务器都要运行这些协议。
- 电子邮件的用户代理就是用户与电子邮件系统的接口，它向用户提供一个很友好的视窗界面来发送和接收邮件。
- 从用户代理把邮件传送到邮件服务器，以及在邮件服务器之间的传送，都要使用 SMTP 协议。但用户代理从邮件服务器读取邮件时，则要使用 POP3（或 IMAP）协议。
- 基于万维网的电子邮件使用户能够利用浏览器收发电子邮件。用户浏览器和邮件服务器之间的邮件传送使用 HTTP 协议，而在邮件服务器之间邮件的传送仍然使用 SMTP 协议。
- 简单网络管理协议 SNMP 由三部分组成，即(1) SNMP 本身，负责读取和改变各代理中的对象名及其状态数值；(2) 管理信息结构 SMI，定义命名对象和定义对象类型（包括范围和长度）的通用规则，以及把对象和对象的值进行编码的基本编码规则 BER；(3) 管理信息库 MIB，在被管理的实体中创建了命名对象，并规定了其类型。

- 系统调用接口是应用进程的控制权和操作系统的控制权进行转换的一个接口，又称为应用编程接口 API。API 就是应用程序和操作系统之间的接口。
- 套接字是应用进程和运输层协议之间的接口，是应用进程为了获得网络通信服务而与操作系统进行交互时使用的一种机制。
- 目前 P2P 工作方式下的文件共享在互联网流量中已占据最大的份额，比万维网应用所占的比例大得多。
- BT 是很流行的一种 P2P 应用。BT 采用“最稀有的优先”的技术，可以尽早把最稀有的文件块收集到。此外，凡有当前以最高数据率向某个对等方传送文件块的相邻对等方，该对等方就优先把所请求的文件块传送给这些相邻对等方。这样做的结果是，这些对等方相互之间都能够以令人满意的速率交换文件块。
- 当对等方的数量很大时，采用 P2P 方式下载大文件，要比传统的客户-服务器方式快得多。
- 在 P2P 应用中，广泛使用的索引和查找技术是分布式散列表 DHT。

习题

- 6-01 互联网的域名结构是怎样的？它与目前的电话网的号码结构有何异同之处？
- 6-02 域名系统的主要功能是什么？域名系统中的本地域名服务器、根域名服务器、顶级域名服务器以及权限域名服务器有何区别？
- 6-03 举例说明域名转换的过程。域名服务器中的高速缓存的作用是什么？
- 6-04 设想有一天整个互联网的 DNS 系统都瘫痪了（这种情况不大会出现），试问还有可能给朋友发送电子邮件吗？
- 6-05 文件传送协议 FTP 的主要工作过程是怎样的？为什么说 FTP 是带外传送控制信息？主进程和从属进程各起什么作用？
- 6-06 简单文件传送协议 TFTP 与 FTP 的主要区别是什么？各用在什么场合？
- 6-07 远程登录 TELNET 的主要特点是什么？什么叫作虚拟终端 NVT？
- 6-08 解释以下名词。各英文缩写词的原文是什么？
WWW, URL, HTTP, HTML, CGI, 浏览器, 超文本, 超媒体, 超链, 页面, 活动文档, 搜索引擎。
- 6-09 假定一个超链从一个万维网文档链接到另一个万维网文档时，由于万维网文档上出现了差错而使得超链指向一个无效的计算机名字。这时浏览器将向用户报告什么？
- 6-10 假定要从已知的 URL 获得一个万维网文档。若该万维网服务器的 IP 地址开始时并不知道。试问：除 HTTP 外，还需要什么应用层协议和运输层协议？
- 6-11 你所使用的浏览器的高速缓存有多大？请进行一个实验：访问几个万维网文档，然后将你的计算机与网络断开，然后再回到你刚才访问过的文档。你的浏览器的高速缓存能够存放多少个页面？
- 6-12 什么是动态文档？试举出万维网使用动态文档的一些例子。
- 6-13 浏览器同时打开多个 TCP 连接进行浏览的优缺点如何？请说明理由。
- 6-14 请判断以下论述的正误，并简述理由。

- (1) 用户点击某网页, 该网页有 1 个文本文件和 3 个图片。此用户可以发送一个请求就可以收到 4 个响应报文。
 - (2) 有以下两个不同的网页: `www.abc.com/m1.html` 和 `www.abc.com/m2.html`。用户可以使用同一个 HTTP/1.1 持续连接传送对这两个网页的请求和响应。
 - (3) 在客户与服务器之间的非持续连接, 只需要用一个 TCP 报文段就能够装入两个不同的 HTTP 请求报文。
 - (4) 在 HTTP 响应报文中的主体实体部分永远不会是空的。
- 6-15** 假定你在浏览器上点击一个 URL, 但这个 URL 的 IP 地址以前并没有缓存在本地主机上。因此需要用 DNS 自动查找和解析。假定要解析到所要找的 URL 的 IP 地址共经过 n 个 DNS 服务器, 所经过的时间分别为 $RTT_1, RTT_2, \dots, RTT_n$ 。假定从要找的网页上只需要读取一个很小的图片 (即忽略这个小图片的传输时间)。从本地主机到这个网页的往返时间是 RTT_w 。试问从点击这个 URL 开始, 一直到本地主机的屏幕上出现所读取的小图片, 一共要经过多长时间?
- 6-16** 在上题中, 假定同一台服务器的 HTML 文件中又链接了三个非常小的对象。若忽略这些对象的发送时间, 试计算客户点击读取这些对象所需的时间。
- (1) 没有并行 TCP 连接的非持续 HTTP;
 - (2) 使用并行 TCP 连接的非持续 HTTP;
 - (3) 流水线方式的持续 HTTP。
- 6-17** 在浏览器中应当有几个可选解释程序。试给出一些可选解释程序的名称。
- 6-18** 一个万维网网点有 1000 万个页面, 平均每个页面有 10 个超链。读取一个页面平均要 100 ms。请问: 要检索整个网点所需的最少时间是多少?
- 6-19** 搜索引擎可分为哪两种类型? 各有什么特点?
- 6-20** 试述电子邮件的最主要的组成部件。用户代理 UA 的作用是什么? 没有 UA 行不行?
- 6-21** 电子邮件的信封和内容在邮件的传送过程中起什么作用? 和用户的关系如何?
- 6-22** 电子邮件的地址格式是怎样的? 请说明各部分的意思。
- 6-23** 试简述 SMTP 通信的三个阶段的过程。
- 6-24** 试述邮局协议 POP 的工作过程。在电子邮件中, 为什么需要使用 POP 和 SMTP 这两个协议? IMAP 与 POP 有何区别?
- 6-25** MIME 与 SMTP 的关系是怎样的? 什么是 quoted-printable 编码和 base64 编码?
- 6-26** 一个二进制文件共 3072 字节长。若使用 base64 编码, 并且每发送完 80 字节就插入一个回车符 CR 和一个换行符 LF, 问一共发送了多少个字节?
- 6-27** 试将数据 11001100 10000001 00111000 进行 base64 编码, 并得出最后传送的 ASCII 数据。
- 6-28** 试将数据 01001100 10011101 00111001 进行 quoted-printable 编码, 并得出最后传送的 ASCII 数据。这样的数据用 quoted-printable 编码后, 其编码开销有多大?
- 6-29** 电子邮件系统需要将人们的电子邮件地址编成目录以便于查找。要建立这种目录应将人名划分为几个标准部分 (例如, 姓、名)。若要形成一个国际标准, 那么必须解决哪些问题?
- 6-30** 电子邮件系统使用 TCP 传送邮件。为什么有时我们会遇到邮件发送失败的情况? 为什么有时对方会收不到我们发送的邮件?

- 6-31 基于万维网的电子邮件系统有什么特点？在传送邮件时使用什么协议？
- 6-32 DHCP 协议用在什么情况下？当一台计算机第一次运行引导程序时，其 ROM 中有没有该主机的 IP 地址、子网掩码或某个域名服务器的 IP 地址？
- 6-33 什么是网络管理？为什么说网络管理是当今网络领域中的热门课题？
- 6-34 解释下列术语：网络元素、被管对象、管理进程、代理进程和管理信息库。
- 6-35 SNMP 使用 UDP 传送报文。为什么不使用 TCP？
- 6-36 为什么 SNMP 的管理进程使用探测掌握全网状态属于正常情况，而代理进程用陷阱向管理进程报告属于较少发生的异常情况？
- 6-37 SNMP 使用哪几种操作？SNMP 在 Get 报文中设置了请求标识符字段，为什么？
- 6-38 什么是管理信息库 MIB？为什么要使用 MIB？
- 6-39 什么是管理信息结构 SMI？它的作用是什么？
- 6-40 用 ASN.1 基本编码规则对以下 4 个数组(SEQUENCE-OF)进行编码。假定每一个数字占用 4 个字节。
2345, 1236, 122, 1236
- 6-41 SNMP 要发送一个 GetRequest 报文，以便向一个路由器获取 ICMP 的 icmpInParmProbs 的值。在 icmp 中变量 icmpInParmProbs 的标号是 5，它是一个计数器，用来统计收到的类型为参数问题的 ICMP 差错报告报文的数目。试给出这个 GetRequest 报文的编码。
- 6-42 对象 tcp 的 OBJECT IDENTIFIER 是什么？
- 6-43 在 ASN.1 中，IP 地址(IPAddress)的类别是应用类。若 IPAddress = 131.21.14.2，试求其 ASN.1 编码。
- 6-44 什么是应用编程接口 API？它是应用程序和谁的接口？
- 6-45 试举出常用的几种系统调用的名称，说明它们的用途。
- 6-46 图 6-37 表示了各应用协议在层次中的位置。
(1) 简单讨论一下为什么有的应用层协议要使用 TCP 而有的却要使用 UDP？
(2) 为什么 MIME 画在 SMTP 之上？
(3) 为什么路由选择协议 RIP 放在应用层？

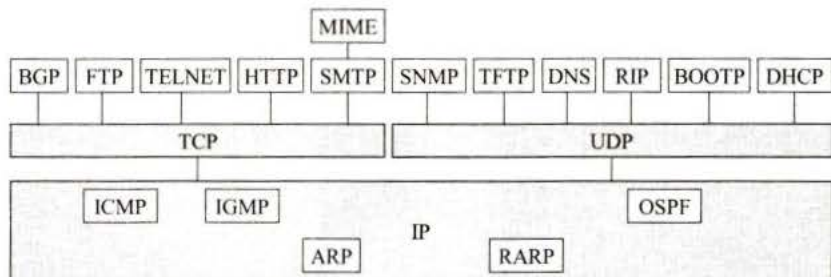


图 6-37 习题 6-46 的图

- 6-47 现在流行的 P2P 文件共享应用程序都有哪些特点？存在哪些值得注意的问题？
- 6-48 使用客户-服务器方式进行文件分发。一台服务器把一个长度为 F 的大文件分发给 N 个对等方。假设文件传输的瓶颈是各计算机（包括服务器）的上传速率 u 。试计算文件分发到所有对等方的最短时间。

- 6-49 重新考虑上题的文件分发任务，但采用 P2P 文件分发方式，并且每个对等方只能在接收完整个文件后才能向其他对等方转发。试计算文件分发到所有 N 个对等方的最短时间。
- 6-50 再重新考虑上题的文件分发任务，但可以把这个非常大的文件划分为一个个非常小的数据块进行分发，即一个对等方在下载完一个数据块后就能向其他对等方转发，并同时可下载其他数据块。不考虑分块增加的控制信息，试计算整个大文件分发到所有对等方的最短时间。
- 6-51 假定某服务器有一文件 $F = 15 \text{ Gbit}$ 要分发给分布在互联网各处的 N 个对等方。服务器上传速率 $u_s = 30 \text{ Mbit/s}$ ，每个对等方的下载速率 $d = 2 \text{ Mbit/s}$ ，上传速率为 $u = 300 \text{ kbit/s}$ 。设(1) $N = 10$ ，(2) $N = 1000$ 。试分别计算在客户-服务器方式下和在 P2P 方式下，该文件分发时间的最小值。

第7章 网络安全

随着计算机网络的发展,网络中的安全问题也日趋严重。当网络的用户来自社会各个阶层与部门时,大量在网络中存储和传输的数据就需要保护。由于计算机网络安全是另一门专业学科,所以本章只对计算机网络安全问题的基本内容进行初步的介绍。

本章最重要的内容是:

- (1) 计算机网络面临的安全性威胁和计算机网络安全的主要问题。
- (2) 对称密钥密码体制和公钥密码体制的特点。
- (3) 鉴别、报文鉴别码、数字签名、证书、证书链的概念。
- (4) 网络层安全协议 IPsec 协议族和运输层安全协议 TLS 的要点。
- (5) 应用层电子邮件的安全措施。
- (6) 系统安全:防火墙与入侵检测。

7.1 网络安全问题概述

本节讨论计算机网络面临的安全性威胁、安全的内容和一般的数据加密模型。

7.1.1 计算机网络面临的安全性威胁

计算机网络的通信面临两大类威胁,即**被动攻击**和**主动攻击**(如图 7-1 所示)。

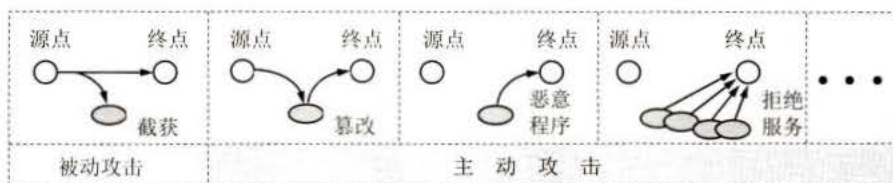


图 7-1 对网络的被动攻击和主动攻击

被动攻击是指攻击者从网络上窃听他人的通信内容。通常把这类攻击称为**截获**。在被动攻击中,攻击者只是观察和分析某一个**协议数据单元 PDU**(这里使用 PDU 这一名词是考虑到所涉及的可能是不同的层次)而不干扰信息流。即使这些数据对攻击者来说是不易理解的,他也可通过观察 PDU 的协议控制信息部分,了解正在通信的协议实体的地址和身份,研究 PDU 的长度和传输的频度,从而了解所交换的数据的某种性质。这种被动攻击又称为**流量分析(traffic analysis)**。在战争时期,通过分析某处出现大量异常的通信量,往往可以发现敌方指挥所的位置。

主动攻击有如下几种最常见的方式。

(1) **篡改** 攻击者故意篡改网络上传送的报文。这里也包括彻底中断传送的报文,甚至是把完全伪造的报文发送给接收方。这种攻击方式有时也称为更改报文流。

(2) **恶意程序** 恶意程序(rogue program)种类繁多,对网络安全威胁较大的主要有以下几种:

- **计算机病毒(computer virus)**, 一种会“传染”其他程序的程序,“传染”是通过修改其他程序来把自身或自己的变种复制进去而完成的。
- **计算机蠕虫(computer worm)**, 一种通过网络的通信功能将自身从一个节点发送到另一个节点并自动启动运行的程序。
- **特洛伊木马(Trojan horse)**, 一种程序, 它执行的功能并非所声称的功能而是某种恶意功能。如一个编译程序除了执行编译任务以外, 还把用户的源程序偷偷地复制下来, 那么这种编译程序就是一种特洛伊木马。计算机病毒有时也以特洛伊木马的形式出现。
- **逻辑炸弹(logic bomb)**, 一种当运行环境满足某种特定条件时执行其他特殊功能的程序。如一个编辑程序, 平时运行得很好, 但当系统时间为 13 日又为星期五时, 它会删去系统中所有的文件, 这种程序就是一种逻辑炸弹。
- **后门入侵(backdoor knocking)**, 是指利用系统实现中的漏洞通过网络入侵系统。就像一个盗贼在夜晚试图闯入民宅, 如果某家住户的房门有缺陷, 盗贼就能乘虚而入。索尼游戏网络(PlayStation Network)在 2011 年被入侵, 导致 7700 万用户的个人信息, 诸如姓名、生日、E-mail 地址、密码等被盗[W-BACKD]。
- **流氓软件**, 一种未经用户允许就在用户计算机上安装运行并损害用户利益的软件, 其典型特征是: 强制安装、难以卸载、浏览器劫持、广告弹出、恶意收集用户信息、恶意卸载、恶意捆绑等。现在流氓软件的泛滥程度已超过了各种计算机病毒, 成为互联网上最大的公害。流氓软件的名字一般都很吸引人, 如某某卫士、某某搜霸等, 因此要特别小心。

上面所说的计算机病毒是狭义的, 也有人把所有的恶意程序泛指为计算机病毒。例如 1988 年 10 月“Morris 病毒”入侵美国互联网, 舆论说该事件是“计算机病毒入侵美国计算机网”, 而计算机安全专家却称之为“互联网蠕虫事件”。

(3) 拒绝服务 DoS (Denial of Service) 指攻击者向互联网上的某个服务器不停地发送大量分组, 使该服务器无法提供正常服务, 甚至完全瘫痪。2000 年 2 月 7 日至 9 日美国几个著名网站遭黑客^①袭击, 使这些网站的服务器一直处于“忙”的状态, 因而无法向发出请求的客户提供服务。这种攻击被称为**拒绝服务**。又如在 2014 年圣诞节, 索尼游戏网(PlayStation Network)和微软游戏网(Microsoft Xbox Live)被黑客攻击后瘫痪, 估计有 1.6 亿用户受到影响[W-DOS]。

若从互联网上的成百上千个网站集中攻击一个网站, 则称为**分布式拒绝服务 DDoS (Distributed Denial of Service)**。有时也把这种攻击称为**网络带宽攻击或连通性攻击**。

2018 年 2 月, 6 日世界标准时间零时, 亚太地区许多计算机同时向根服务器系统发动袭击, 企图使之瘫痪。它们每秒传送的数据量相当于服务器每分钟要接收 75 万封电子邮件。结果至少有 6 个根服务器系统受到影响, 两个破坏严重。有分析认为攻击来自韩国, 但

① 注: 黑客(hacker)是指精通计算机编程的高手, 他们能够通过专门的技术手段进入某些据称是相当安全的计算机系统中。黑客一般可分为两大类。一类是蓄意搞破坏或盗窃别人计算机中数据信息的坏人, 而另一类则是专门研究计算机系统安全性的好人。例如, 银行发行的信用卡必须十分安全, 但这种信用卡的安全性在公开发行之前却无从知晓。这时就要请专门研究计算机安全的黑客对信用卡进行攻击实验。如黑客在努力尝试后仍无法攻破, 则可认为该信用卡至少在目前是相对安全的。

ICANN 不认为黑客一定是韩国人，他可以是任何地点的任何人，只不过是操纵了韩国的计算机而已。

还有其他类似的网络安全问题。例如，在使用以太网交换机的网络中，攻击者向某个以太网交换机发送大量的伪造源 MAC 地址的帧。以太网交换机收到这样的帧，就把这个假的源 MAC 地址写入交换表中（因为交换表中没有这个地址）。由于这种伪造的地址数量太大，因此很快就把交换表填满了，导致以太网交换机无法正常工作（称为**交换机中毒**）。

对于主动攻击，可以采取适当措施加以检测。但对于被动攻击，通常却是检测不出来的。根据这些特点，可得出计算机网络通信安全的目标如下：

(1) 防止析出报文内容和流量分析。

(2) 防止恶意程序。

(3) 检测更改报文流和拒绝服务。

对付被动攻击可采用各种数据加密技术，而对付主动攻击，则需将加密技术与适当的鉴别技术相结合。

7.1.2 安全的计算机网络

人们一直希望能设计出一种安全的计算机网络，但不幸的是，网络的安全性是不可判定的[DENN82]。目前在安全协议的设计方面，主要是针对具体的攻击设计安全的通信协议。但如何保证所设计出的协议是安全的？这可以使用两种方法。一种是用形式化方法来证明，另一种是用经验来分析协议的安全性。形式化证明的方法是人们所希望的，但一般意义上的协议安全性也是不可判定的，只能针对某种特定类型的攻击来讨论其安全性。对于复杂的通信协议的安全性，形式化证明比较困难，所以主要采用人工分析的方法来找漏洞。对于简单的协议，可通过限制入侵者的操作（即假定入侵者不会进行某种攻击）来对一些特定情况进行形式化的证明，当然，这种方法有很大的局限性。

根据上一节所述的各种安全性威胁，不难看出，一个安全的计算机网络应设法达到以下四个目标：

1. 机密性

机密性（或私密性）就是只有信息的发送方和接收方才能懂得所发送信息的内容，而信息的截获者则看不懂所截获的信息。显然，机密性是网络安全通信最基本的要求，也是对付被动攻击所必须具备的功能。通常可简称为**保密**。尽管计算机网络安全并不仅仅依靠机密性，但不能提供机密性的网络肯定是不安全的。为了使网络具有机密性，需要使用各种密码技术。

2. 端点鉴别

安全的计算机网络必须能够鉴别信息的发送方和接收方的真实身份。网络通信和面对面的通信差别很大。现在频繁发生的网络诈骗，在许多情况下，就是由于在网络上不能鉴别出对方的真实身份。当我们收到一封电子邮件时，发信人也可能并不是邮件上所署名的那个人。当我们进行网上购物时，卖家也有可能是犯罪分子假冒的商家。不能解决这个问题，就不能认为网络是安全的。端点鉴别在对付主动攻击时是非常重要的。

3. 信息的完整性

即使能够确认发送方的身份是真实的，并且所发送的信息都是经过加密的，我们依然不能认为网络是安全的。还必须确认所收到的信息都是完整的，也就是信息的内容没有被人篡改过。保证信息的完整性在应对主动攻击时也是必不可少的。信息的完整性和机密性是两个不同的概念。例如，商家向公众发布的商品广告当然不需要保密，但如果广告在网络上传送时被人恶意删除或添加了一些内容，那么就可能对商家造成很大的损失。

实际上，信息的完整性与端点鉴别往往是不可分割的。假定你准确知道报文发送方的身份没有错（即通过了端点鉴别），但收到的报文却已被人篡改过（即信息不完整），那么这样的报文显然是没有用处的。因此，在谈到“鉴别”时，有时是同时包含了端点鉴别和报文的完整性。也就是说，既鉴别发送方的身份，又鉴别报文的完整性。

4. 运行的安全性

现在的机构与计算机网络的关系越密切，就越要重视计算机网络运行的安全性。上一节介绍的恶意程序和拒绝服务的攻击，即使没有窃取到任何有用的信息，也能够使受到攻击的计算机网络不能正常运行，甚至完全瘫痪。因此，确保计算机系统运行的安全性，也是非常重要的工作。对于一些要害部门，这点尤为重要。

访问控制(access control)对计算机系统的安全性非常重要。必须对访问网络的权限加以控制，并规定每个用户的访问权限。由于网络是个非常复杂的系统，其访问控制机制比操作系统的访问控制机制更复杂（尽管网络的访问控制机制是建立在操作系统的访问控制机制之上的），尤其在安全要求更高的**多级安全(multilevel security)**情况下更是如此。

7.1.3 数据加密模型

一般的数据加密模型如图 7-2 所示。用户 A 向 B 发送明文 X，但通过**加密算法 E** 运算后，就得出密文 Y。

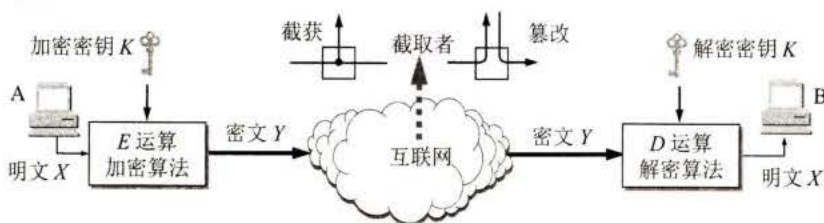


图 7-2 一般的数据加密模型

图中所示的加密和解密用的**密钥 K (key)**是一串秘密的字符串（即比特串）。公式(7-1)就是明文通过加密算法变成密文的一般表示方法。

$$Y = E_K(X) \quad (7-1)$$

在传送过程中可能出现密文的**截取者（或攻击者、入侵者）**。公式(7-2)表示接收端利用解密算法 D 运算和解密密钥 K，解出明文 X。解密算法是加密算法的逆运算。在进行解密运算时，如果不使用事先约定好的密钥就无法解出明文。

$$D_K(Y) = D_K(E_K(X)) = X \quad (7-2)$$

这里我们假定加密密钥和解密密钥都是一样的。但实际上它们可以是不一样的（即使不一样，这两个密钥也必然有某种相关性）。密钥通常由密钥中心提供。当密钥需要向远地传送时，一定要通过另一个安全信道。

密码编码学(cryptography)是密码体制的设计学，而**密码分析学(cryptanalysis)**则是在未知密钥的情况下从密文推演出明文或密钥的技术。密码编码学与密码分析学合起来即为**密码学(cryptology)**。

如果不论截取者获得了多少密文，但在密文中都没有足够的信息来唯一地确定出对应的明文，则这一密码体制称为**无条件安全的**，或称为**理论上是不可破的**。在无任何限制的条件下，目前几乎所有实用的密码体制均是可破的。因此，人们关心的是要研制出**在计算上（而不是在理论上）是不可破的密码体制**。如果一个密码体制中的密码，不能在一定时间内被可以使用的计算资源破译，则这一密码体制称为**在计算上是安全的**。

早在几千年前人类就已经有了通信保密的思想和方法。直到 1949 年，信息论创始人香农(C. E. Shannon)发表著名文章[SHAN49]，论证了一般经典加密方法得到的密文几乎都是可破的。密码学的研究曾面临着严重的危机。但从 20 世纪 60 年代起，随着电子技术、计算技术的迅速发展以及结构代数、可计算性和计算复杂性理论等学科的研究，密码学又进入了一个新的发展时期。在 20 世纪 70 年代后期，美国的数据加密标准 DES (Data Encryption Standard) 和公钥密码体制（public key crypto-system，又称为公开密钥密码体制）的出现，成为近代密码学发展史上的两个重要里程碑。

7.2 两类密码体制

7.2.1 对称密钥密码体制

所谓**对称密钥密码体制**，即加密密钥与解密密钥都使用相同密钥的密码体制。例如图 7-2 所示的情况，通信的双方使用的就是对称密钥。

数据加密标准 DES 属于对称密钥密码体制。它由 IBM 公司研制出，于 1977 年被美国定为联邦信息标准后，在国际上引起了极大的重视。ISO 曾将 DES 作为数据加密标准。

DES 是一种分组密码。在加密前，先对整个的明文进行分组。每一个组为 64 位长的二进制数据。然后对每一个 64 位二进制数据进行加密处理，产生一组 64 位密文数据。最后将各组密文串接起来，即得出整个的密文。使用的密钥占有 64 位（实际密钥长度为 56 位，外加 8 位用于奇偶校验）。

DES 的机密性仅取决于对密钥的保密，而算法是公开的。DES 的问题是它的密钥长度。56 位长的密钥意味着共有 2^{56} 种可能的密钥，也就是说，共有约 7.6×10^{16} 种密钥。假设一台计算机 $1\ \mu\text{s}$ 可执行一次 DES 加密，同时假定平均只需搜索密钥空间的一半即可找到密钥，那么破译 DES 要超过 1000 年。

然而芯片的发展出乎意料地快。不久，56 位 DES 已不再被认为是安全的。

对于 DES 56 位密钥的问题，学者们提出了三重 DES (Triple DES 或记为 3DES) 的方案，把一个 64 位明文用一个密钥加密，再用另一个密钥解密，然后再使用第一个密钥加密，即

$$Y = \text{DES}_{K1}(\text{DES}_{K2}^{-1}(\text{DES}_{K1}(X)))$$

这里, X 是明文, Y 是密文, K_1 和 K_2 分别是第一个和第二个密钥, $DES_{K_1}(\cdot)$ 表示用密钥 K_1 进行 DES 加密, 而 $DES_{K_2}^{-1}(\cdot)$ 表示用密钥 K_2 进行 DES 解密。

这种三重 DES 曾广泛用于网络、金融、信用卡等系统。

在 DES 之后, 1997 年美国标准与技术协会(NIST), 对一种新的加密标准即高级加密标准 AES (Advanced Encryption Standard) 进行遴选, 最后由两位年轻比利时学者 Joan Daemen 和 Vincent Rijmen 提交的 Rijndael 算法被选中, 在 2001 年正式成为 NIST 的加密标准。在 2002 年成为美国政府加密标准。现在 AES 也是 ISO/IEC 18033-3 标准。

AES 是一种分组密码, 分组长度为 128 位。AES 有三种加密标准, 其密钥分别为 128 位、192 位和 256 位, 加密步骤相当复杂, 运算速度比 3DES 快得多, 且安全性也大大加强。在 2001 年, NIST 曾有一个大致的估计, 就是假定有一台高速计算机, 仅用 1 秒钟就能够破译 56 位的 DES (也就是采用穷举法, 在 1 秒钟内能够把 DES 所有的 2^{56} 个密钥逐个进行解密运算一遍), 那么要破译 128 位的 AES, 就需要 10^{12} 年! 但是计算机运算速度的提高是很难预测的。因此美国国家安全局 NSA 认为, 对于最高机密信息 (这类信息必须保证数十年以上的安全性) 的传递, 至少需要 192 或 256 位的密钥长度。

到 2020 年 5 月为止, 尚未见到能够成功破解 AES 密码系统的报道。有人认为, 要破解 AES 可能需要在数学上出现非常重大的突破。

7.2.2 公钥密码体制

公钥密码体制的概念是由斯坦福(Stanford)大学的研究人员 Diffie 与 Hellman 于 1976 年提出的[DIFF76]。公钥密码体制使用不同的加密密钥与解密密钥。这种加密体制又称为非对称密钥密码体制。

公钥密码体制的产生主要有两个方面的原因, 一是由于对称密钥密码体制的密钥分配问题, 二是由于对数字签名的需求。

在对称密钥密码体制中, 加解密的双方使用的是相同的密钥。但怎样才能做到这一点呢? 一种是事先约定, 另一种是用信使来传送。在高度自动化的大型计算机网络中, 用信使来传送密钥显然是不合适的。如果事先约定密钥, 就会给密钥的管理和更换带来极大的不便。若使用高度安全的密钥分配中心 KDC (Key Distribution Center), 也会使得网络成本增加。

对数字签名的强烈需要也是产生公钥密码体制的一个原因。在许多应用中, 人们需要对纯数字的电子信息进行签名, 表明该信息确实是某个特定的人产生的。

公钥密码体制提出不久, 人们就找到了三种公钥密码体制。目前最著名的是由美国三位科学家 Rivest, Shamir 和 Adleman 于 1976 年提出并在 1978 年正式发表的 RSA 体制, 它是一种基于数论中的大数分解问题的体制[RIVE78]。

在公钥密码体制中, 加密密钥 PK (Public Key, 即公钥) 是向公众公开的, 而解密密钥 SK (Secret Key, 即私钥或密钥) 则是需要保密的。加密算法 E 和解密算法 D 也都是公开的。

公钥密码体制的加密和解密过程有如下特点:

(1) 密钥对产生器产生出接收者 B 的一对密钥: 加密密钥 PK_B 和解密密钥 SK_B 。发送者 A 所用的加密密钥 PK_B 就是接收者 B 的公钥, 它向公众公开。而 B 所用的解密密钥 SK_B 就是接收者 B 的私钥, 对其他人都保密。

(2) 发送者 A 用 B 的公钥 PK_B 通过 E 运算对明文 X 加密, 得出密文 Y , 发送给 B。

$$Y = E_{PK_B}(X) \quad (7-3)$$

B 用自己的私钥 SK_B 通过 D 运算进行解密，恢复出明文，即

$$D_{SK_B}(Y) = D_{SK_B}(E_{PK_B}(X)) = X \quad (7-4)$$

(3) 虽然在计算机上可以容易地产生成对的 PK_B 和 SK_B ，但从已知的 PK_B 实际上不可能推导出 SK_B ，即从 PK_B 到 SK_B 是“计算上不可能的”。这就是说，除了 B 以外，其他任何人都无法解密出明文 X 。

(4) 虽然公钥可用来加密，但却不能用来解密，即

$$D_{PK_B}(E_{PK_B}(X)) \neq X \quad (7-5)$$

(5) 先后对 X 进行 D 运算和 E 运算或进行 E 运算和 D 运算，结果都是一样的：

$$E_{PK_B}(D_{SK_B}(X)) = D_{SK_B}(E_{PK_B}(X)) = X \quad (7-6)$$

请注意，通常都是先加密然后再解密。但仅从运算的角度看， D 运算和 E 运算的先后顺序则可以是任意的。对某个报文进行 D 运算，并不表明是要对其解密。

图 7-3 给出了用公钥密码体制进行加密的过程。



图 7-3 公钥密码体制

公开密钥与对称密钥在使用通信信道方面有很大的不同。在使用对称密钥时，由于双方使用同样的密钥，因此在通信信道上可以进行一对一的双向保密通信，每一方既可用此密钥加密明文，并发送给对方，也可接收密文，用同一密钥对密文解密。这种保密通信仅限于持有此密钥的双方（如再有第三方就不保密了）。但在使用公钥密码体制时，在通信信道上可以是多对一的单向保密通信。例如在图 7-3 中，可以有很多人同时持有 B 的公钥，并各自用此公钥对自己的报文加密后发送给 B。只有 B 才能够用其私钥对收到的多个密文一一进行解密。但使用这对密钥进行反方向的保密通信则是不行的。在现实生活中，这种多对一的单向保密通信是很常用的。例如，在网购时，很多顾客都向同一个网站发送各自的信用卡信息，就属于这种情况。

请注意，任何加密方法的安全性取决于密钥的长度，以及攻破密文所需的计算量，而不是简单地取决于加密的体制（公钥密码体制或传统加密体制）。我们还要指出，公钥密码体制并没有使传统密码体制被弃用，因为目前公钥加密算法的开销较大，在可见的将来还不会放弃传统加密方法。

7.3 鉴别

7.3.1 报文鉴别

在网络的应用中，**鉴别(authentication)**是网络安全中一个很重要的问题。鉴别和加密是不相同的概念。鉴别的内容有二。一是要**鉴别发信者**，即验证通信的对方的确是自己所要通信的对象，而不是其他的冒充者。这就是**实体鉴别**。实体可以是发信的人，也可以是一个进程（客户或服务器）。因此这也常称为**端点鉴别**。二是要**鉴别报文的完整性**，即对方所传送的报文没有被他人篡改过。至于报文是否需要加密，则是与“鉴别”性质不同的问题。有的报文需要加密（这要另找措施），但许多报文并不需要加密。

请注意，鉴别与**授权(authorization)**也是不同的概念。授权涉及的问题是：所进行的过程是否被允许（如是否可以对某文件进行读或写）。

不过有时常用**报文鉴别**一词包含上述鉴别的两个内容，既鉴别报文的发送者，也鉴别报文的完整性。

下面分别讨论报文鉴别与实体鉴别的特点。

1. 用数字签名进行鉴别（原理）

我们知道，书信或文件可根据亲笔签名或印章来鉴别其真实性。但在计算机网络中传送的报文，则可使用**数字签名**进行鉴别。下面就介绍数字签名的原理。

为了进行**数字签名**，A 用其私钥 SK_A 对报文 X 进行 D 运算（如图 7-4 所示）。 D 运算本来叫作解密运算。可是，还没有加密怎么就进行解密呢？其实 D 运算只是把报文变换为某种不可读的密文（因此有时也说成 A 用其私钥对报文加密，但这样说不准确）。在图 7-4 中我们使用“ D 运算”而不是“解密运算”，就是为了避免产生这种误解。A 把经过 D 运算得到的密文传送给 B。B 为了**核实签名**，用 A 的公钥进行 E 运算，还原出明文 X 。请注意，任何人用 A 的公钥 PK_A 进行 E 运算后都可以得出 A 发送的明文。可见图 7-4 所示的通信方式并非为了保密，而是为了进行签名和核实签名，即确认此明文的确是 A 发送的。



图 7-4 用数字签名进行鉴别

下面讨论一下为什么数字签名具有鉴别报文的功能。

因为除 A 外没有别人持有 A 的私钥 SK_A ，所以除 A 外没有别人能产生密文 $D_{SK_A}(X)$ 。这样，B 确信报文 X 是 A 签名发送的。这就鉴别了报文的发送者。同理，其他人如果篡改过报文，但由于无法得到 A 的私钥 SK_A 对篡改后的报文进行 D 运算，那么 B 对收到的报文进行核实签名的 E 运算后，将会得出不可读的明文，因而不会被欺骗。这样就保证了报文的完整性。

数字签名还有另一功能，就是发送者事后不能抵赖对报文的签名。这叫作**不可否认**。

若 A 要抵赖曾发送报文给 B, B 可把 X 及 $D_{SK_A}(X)$ 出示给进行公证的第三者。第三者很容易用 PK_A 去证实 A 确实发送 X 给 B。

以上这三项功能的关键都在于没有其他人能够持有 A 的私钥 SK_A 。

但数字签名仅对报文进行了签名, 对报文 X 本身却未保密。因为截获到密文 $D_{SK_A}(X)$ 并知道发送者身份的任何人, 若通过某种手段获得了发送者的公钥 PK_A , 就能解出报文的内容。如果用图 7-5 所示的方法, 就可同时实现保密通信和数字签名。图中 SK_A 和 SK_B 分别为 A 和 B 的私钥, 而 PK_A 和 PK_B 分别为 A 和 B 的公钥。请注意, 在许多情况下, 我们往往强调的是使用何种密钥进行运算, 这时的表达方式可简单些。例如, “用 A 的私钥对明文 X 进行签名” 可记为 $SK_A(X)$ 。若 “再用 B 的公钥对此签名进行加密”, 则可记为 $PK_B(SK_A(X))$, 而不必深究使用的是 D 运算还是 E 运算。

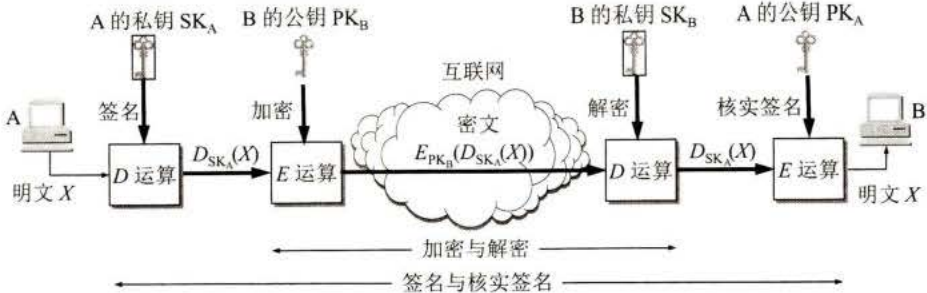


图 7-5 可保证机密性的数字签名

如图 7-5 所示的可保证机密性的数字签名方法, 虽然在理论上是正确的, 但很难用于现实生活中。因此这一节的小标题后边有“原理”二字。这是因为要对报文(可能很长的报文)先后要进行两次 D 运算和两次 E 运算, 这种运算量太大, 要花费非常多的计算机 CPU 时间, 在很多情况下是无法令人接受的。因此目前对网络上传送的大量报文, 普遍都使用开销小得多的对称密钥加密。要实现数字签名当然必须使用公钥密码, 但一定要设法减小公钥密码算法的开销。这就要使用后面几个小节所讨论的密码散列函数和报文鉴别码。

2. 密码散列函数

散列函数(又称为杂凑函数, 或哈希函数) 在计算机领域中使用得很广泛。密码学对散列函数有非常高的要求, 因此符合密码学要求的散列函数又常称为**密码散列函数**(cryptographic hash function)。以后在不致产生错误概念时, 我们也常把密码散列函数简称为散列函数。具体说来, 密码散列函数 $H(X)$ 应具有以下四个特点:

(1) 虽然散列函数的输入报文 X 的长度不受限制, 但计算出的结果 $H(X)$ 的长度则应是较短的和固定的。散列函数的输出 $H(X)$ 又称为**散列值**, 或**散列**。散列函数采用确定算法, 因此相同的输入必定得出相同的输出。虽然密码散列函数相当复杂, 但利用计算机, 散列函数的运算还是相当快的。

(2) 散列函数的输入和输出的关系是多对一的。若散列值 $H(X)$ 的长度为 128 位, 那么输出散列值只有 2^{128} 个有限多的可能值 (2^{128} 与 IPv6 的地址数一样大, 是个很大的数值)。然而我们的输入报文 X 却有无限多的取值。可见必然会出现不同输入却产生相同输出的碰撞现象。精心挑选的密码散列函数应当非常不易发生碰撞, 即应具有很好的**抗碰撞性**。

(3) 若给出散列值 $H(X)$, 则无人能找出输入报文 X 。也就是说, 散列函数是一种单向函数(one-way function), 即逆向变换是不可能的(如图 7-6 所示)。

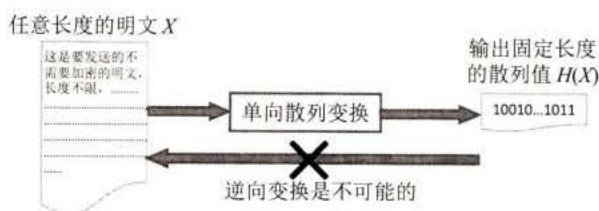


图 7-6 密码散列函数是单向的

上述特点的另一种表述方法是:

若已知 X 和 $H(X)$, 则没有人能够找到 Y ($Y \neq X$), 使得 $H(Y) = H(X)$ 。

但下面我们要讲到, 关于这方面的研究, 后来已有了一些新的进展。

(4) 好的密码散列函数还具有这样一些特性: 散列函数输出的每一个比特, 都与输入的每一个比特有关; 哪怕仅改动输入的一个比特, 输出也会相差极大; 散列函数的运算包括许多非线性运算。

通过许多学者的不断努力, 已经设计出一些实用的密码散列函数(或称为散列算法), 其中最出名的就是 MD5 和 SHA-1。MD 就是 Message Digest 的缩写, 意思是报文摘要。MD5 是报文摘要的第 5 个版本。

报文摘要算法 MD5 公布于 RFC 1321 (1991 年), 并获得了非常广泛的应用。MD5 的设计者 Rivest 曾提出一个猜想, 即根据给定的 MD5 报文摘要代码, 要找出一个与原来报文有相同报文摘要的另一报文, 其难度在计算上几乎是不可能的。但在 2004 年, 中国学者王小云^①发表了轰动世界的密码学论文, 证明可以用系统的方法找出一对报文, 这对报文具有相同的 MD5 报文摘要[W-WANG], 而这仅需 15 分钟, 或不到 1 小时。于是, MD5 的安全性就产生了动摇。随后, 又有许多学者开发了对 MD5 实际的攻击。于是 MD5 最终被另一种叫作安全散列算法 SHA (Secure Hash Algorithm) 的标准所取代。

下面仍以 MD5 为例来介绍报文摘要。这主要是考虑到目前新的散列函数(如 SHA-2)是从 MD5 发展而来的。对于有兴趣研究散列函数的读者, MD5 是个很好的出发点。

MD5 算法的大致过程如下:

(1) 先把任意长的报文按模 2^{64} 计算其余数 (64 位), 追加在报文的后面。

(2) 在报文和余数之间填充 1~512 位, 使得填充后的总长度是 512 的整数倍。填充的首位是 1, 后面都是 0。

(3) 把追加和填充后的报文分割为许多 512 位的数据块, 每个 512 位的报文数据再分成 4 个 128 位的数据块依次送到不同的散列函数进行 4 轮计算。每一轮又都按 32 位的小数据块进行复杂的运算。一直到最后计算出 MD5 报文摘要代码 (128 位)。

这样得出的 MD5 报文摘要代码中的每一位都与原来报文中的每一位有关。由此可见, 像 MD5 这样的密码散列函数实际上已是个相当复杂的算法, 而不是简单的函数了。

SHA-1 是由美国标准与技术协会 NIST 提出的一个散列算法系列。SHA-1 和 MD5 相似,

^① 注: 王小云, 女, 1966 年出生, 山东大学和清华大学教授, 中国科学院院士, 主要研究领域为密码学和信息安全学。

但其散列值的长度为 160 位（比 MD5 的 128 位多了 25%）。SHA-1 也是先把输入报文划分为许多 512 位长的数据块，然后经过复杂运算后得出散列值。SHA-1 比 MD5 更安全，但计算起来却比 MD5 要慢些。1995 年发布的新版本 SHA-1 [RFC 3174] 在安全性方面有了很大的改进。

但 SHA-1 后来也被证明其实际安全性并未达到设计要求，并且也曾被王小云教授的研究团队攻破。谷歌也宣布了攻破 SHA-1 的消息。现在 SHA-1 已被另外的两个版本 SHA-2 [RFC 6234] 和 SHA-3 [FIPS 202] 所替代。SHA-2 和 SHA-3 都各有好几种变型。前者有 SHA-224, SHA-256, SHA-384 和 SHA-512，后者有 SHA3-224, SHA3-256, SHA3-384 和 SHA3-512。在上面名称最后的 3 位数字表示散列的位数。这里需要指出，SHA-3 采用了与 SHA-2 完全不同的散列函数。现在许多组织都已纷纷宣布停用 SHA-1。例如，微软于 2017 年 1 月 1 日起停止支持 SHA-1 证书，而以前签发的 SHA-1 证书也必须更换为 SHA-2 证书。

请注意，MD5 或 SHA-1 “被攻破”，是指有人能设法找出具有相同散列值的一对报文。这样就动摇了 MD5 或 SHA-1 的安全性。但是，密码学家目前尚无法把一个任意已知的报文 X ，篡改为具有同样 MD5 或 SHA-1 散列值的另一报文 Y 。

3. 用报文鉴别码实现报文鉴别

下面进一步讨论怎样使用散列函数来实现报文鉴别。

下面给出的三个简单步骤，给出鉴别报文的初步概念。

(1) 用户 A 首先根据自己的明文 X 计算出散列 $H(X)$ （例如，使用 MD5）。为简单起见，我们把得出的散列 $H(X)$ 记为 H 。

(2) 用户 A 把散列 H 拼接在明文 X 的后面，生成了扩展的报文 (X, H) ，然后发送给 B。

(3) 用户 B 收到了这个扩展的报文 (X, H) 。因为散列的长度 H 是早已知道的固定值，因此很容易把收到的散列 H 和明文 X 分离开。B 通过散列函数的运算，计算出所收到的明文 X 的散列 $H(X)$ 。若 $H(X) = H$ ，则 B 就认为所收到的明文是 A 发送过来的。

但上述做法实际上是不可行的。设想某个入侵者创建了一个伪造的报文 M ，然后也用同样的方法计算出其散列 $H(M)$ ，并且冒充 A 把拼接有散列的扩展报文发送给 B。B 收到扩展的报文 $(M, H(M))$ 后，按照上面步骤 (3) 的方法进行验证，发现一切都是正常的，就会误认为所收到的伪造报文就是 A 发送的。

因此，必须设法对上述的攻击进行防范。解决的办法可以是：A 把双方共享的密钥 K （ K 就是一串不太长的字符串）拼接到报文 X 后，进行散列运算（如图 7-7 所示）。散列运算得出的结果为固定长度的 $H(X + K)$ ，称为报文鉴别码 MAC (Message Authentication Code)。请注意：局域网中使用的媒体接入控制 MAC 正好也使用这三个字母，因此在看到缩写词 MAC 时应注意上下文。A 把报文鉴别码 MAC 拼接在报文 X 后面，得到扩展的报文，发送给 B。我们注意到，共享密钥 K 并没有出现在网上传送的扩展的报文中。

B 收到扩展的报文后，把报文鉴别码 MAC 与报文 X 进行分离。B 再用同样的密钥 K 与报文 X 拼接，进行散列运算，把得出的结果 $H(X + K)$ 与分离出的报文鉴别码 MAC 进行比较。如相等，就可确认收到的报文 X 的确是 A 发送的。只要入侵者不掌握密钥 K ，就无法伪造 A 的报文鉴别码 MAC，因而无法伪造 A 发送的报文。像这样的报文鉴别码称为数字签名，或数字指纹。图 7-7 所示的过程就是 A 对报文进行了签名，而 B 对报文进行了鉴别。

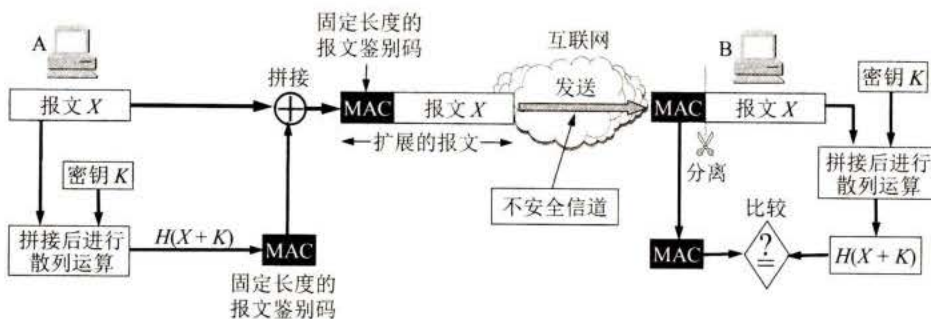


图 7-7 用报文鉴别码 MAC 鉴别报文

图 7-7 所示的鉴别过程并没有执行加密算法，只是在计算散列值时在报文后面拼接了密钥，因此这种鉴别报文的方法消耗的计算资源很少，但却能有效地保护报文的完整性。

在许多有关鉴别的文献中，常常看到在 MAC 前面加上一个 H 的写法，即 HMAC (Hashed MAC)。MAC 与 HMAC 的区别如图 7-8 所示[PETE12, 第 646 页]。前面图 7-7 所示的 MAC 实际上就是 HMAC。计算 HMAC 是规定把密钥 K 拼接在明文后面，然后使用密码散列算法对其进行运算，得出的散列值就是 HMAC。但在计算 MAC 时则不一定这样做。首先，密钥 K 不一定非要拼接在明文的后面，只要把密钥 K 作为一个计算 MAC 的参数即可。其次，可以有多种计算 MAC 的算法，不一定非要使用严格的密码散列算法。在 RFC 2104 中，对各种不同情况下 HMAC 的计算方法都有着详细的规定。但在本书中，为了方便，对 MAC 和 HMAC 可视为同义词。

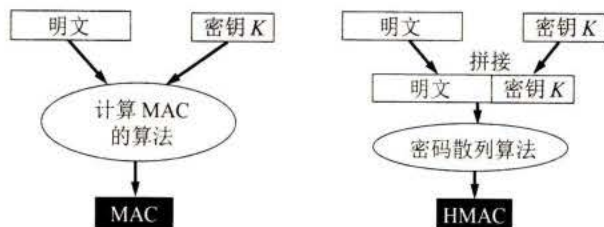


图 7-8 MAC 与 HMAC 的区别

上述这种鉴别报文的方法还有一些问题有待解决。例如，采用怎样安全有效的方法来分发通信双方共享的密钥 K ？另一种可行的方法是采用公钥系统。我们用图 7-9 来说明。

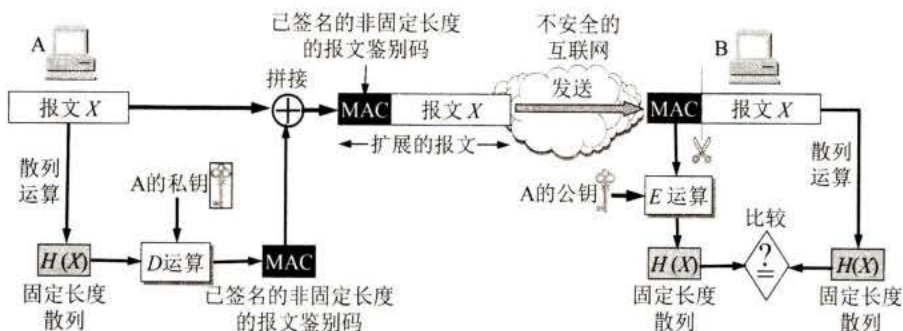


图 7-9 使用已签名的报文鉴别码对报文鉴别

用户 A 对报文 X 进行散列运算，得出固定长度的散列 $H(X)$ 。用自己的私钥对 $H(X)$ 进行 D 运算（也可以说成是用私钥进行加密），得出已签名的但非固定长度的报文鉴别码 MAC。

请注意，这里没有对报文 X 进行加密，而是对很短的散列 $H(X)$ 进行 D 运算，因此这种运算仍然是很快的。A 把已签名的非固定长度的报文鉴别码 MAC，拼接在报文 X 后面，构成扩展的报文发送给 B。

B 收到扩展的报文后，先进行报文分离。虽然 B 不知道已签名的报文鉴别码的长度，但由于报文 X 是明文，其结束处可以设有标记，因此分离出 MAC 不困难。分离后，B 对报文 X 进行散列函数运算，同时用 A 的公钥对分离出的已签名的报文鉴别码 MAC 进行 E 运算（也可以说成是用公钥进行解密）。最后对这两个运算结果 $H(X)$ 进行比较。如相等，就说明一切正确。由于入侵者没有 A 的私钥，因此不可能伪造出 A 发出的报文。这里我们假定 B 事先知道 A 的公钥。

不难看出，采用这种方法得到的扩展的报文，不仅是不可伪造的，也是不可否认的。图 7-9 所示的过程，可简称为：“A 用自己的私钥进行签名，B 用 A 的公钥进行鉴别”。

7.3.2 实体鉴别

实体鉴别和报文鉴别不同。报文鉴别是对每一个收到的报文都要鉴别报文的发送者，而实体鉴别是在系统接入的全部持续时间内对和自己通信的对方实体只需验证一次。

最简单的实体鉴别过程如图 7-10 所示。A 向远端的 B 发送带有自己身份 A（例如，A 的姓名）和口令的报文，并且使用双方约定好的共享对称密钥 K_{AB} 进行加密。B 收到此报文后，用共享对称密钥 K_{AB} 进行解密，从而鉴别了实体 A 的身份。



图 7-10 仅使用对称密钥传送鉴别实体身份的报文

然而这种简单的鉴别方法具有明显的漏洞。例如，入侵者 C 可以从网络上截获 A 发给 B 的报文，C 并不需要破译这个报文（因为破译可能很费时间），而是直接把这个由 A 加密的报文发送给 B，使 B 误认为 C 就是 A；然后 B 就向伪装成 A 的 C 发送许多本来应当发给 A 的报文。这就叫作重放攻击(replay attack)。C 甚至还可以截获 A 的 IP 地址，然后把 A 的 IP 地址冒充为自己的 IP 地址（这叫作 IP 欺骗），使 B 更加容易受骗。

为了对付重放攻击，可以使用不重数(nonce)。不重数就是一个不重复使用的大随机数，即“一次一数”。在鉴别过程中不重数可以使 B 能够把重复的鉴别请求和新的鉴别请求区分开。图 7-11 给出了这个过程。

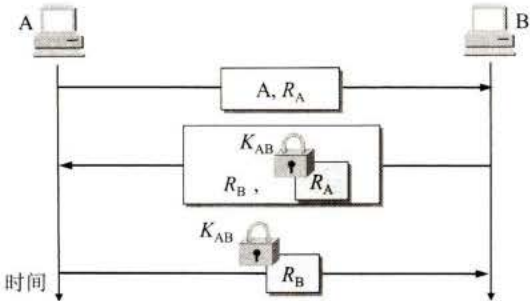


图 7-11 使用不重数进行鉴别

在图 7-11 中, A 首先用明文发送其身份 A 和一个不重数 R_A 给 B。接着, B 响应 A 的查问, 用共享的密钥 K_{AB} 对 R_A 加密后发回给 A, 同时也给出了自己的不重数 R_B 。最后, A 再响应 B 的查问, 用共享的密钥 K_{AB} 对 R_B 加密后发回给 B。这里很重要的一点是 A 和 B 对不同的会话必须使用不同的不重数集。由于不重数不能重复使用, 所以 C 在进行重放攻击时无法重复使用所截获的不重数。

在使用公钥密码体制时, 可以对不重数进行签名鉴别。例如在图 7-11 中, B 用其私钥对不重数 R_A 进行签名后发回给 A。A 用 B 的公钥核实签名, 如能得出自己原来发送的不重数 R_A , 就核实了和自己通信的对方的确是 B。同样, A 也用自己的私钥对不重数 R_B 进行签名后发送给 B。B 用 A 的公钥核实签名, 鉴别了 A 的身份。

公钥密码体制虽然不必在互相通信的用户之间秘密地分配共享密钥, 但仍有受到攻击的可能。让我们看下面的例子。

C 冒充是 A, 发送报文给 B, 说: “我是 A”。

B 选择一个不重数 R_B , 发送给 A, 但被 C 截获了。

C 用自己的私钥 SK_C 冒充是 A 的私钥, 对 R_B 加密, 并发送给 B。

B 向 A 发送报文, 要求对方把解密用的公钥发送过来, 但这报文也被 C 截获了。

C 把自己的公钥 PK_C 冒充是 A 的公钥发送给 B。

B 用收到的公钥 PK_C 对收到的加密的 R_B 进行解密, 其结果当然正确。于是 B 相信通信的对方是 A, 接着就向 A 发送许多敏感数据, 但都被 C 截获了。

然而上述这种欺骗手段不够高明, 因为 B 只要打电话询问一下 A 就能戳穿骗局, 因为 A 根本没有和 B 进行通信。但下面的“中间人攻击”(man-in-the-middle attack)就更加具有欺骗性。图 7-12 是“中间人攻击”的示意图。



图 7-12 中间人攻击

从图 7-12 可看出, A 想和 B 通信, 向 B 发送“我是 A”的报文, 并给出了自己的身份。这个报文被“中间人”C 截获, C 把这个报文原封不动地转发给 B。B 选择一个不重数 R_B 发送给 A, 但同样被 C 截获后也照样转发给 A。

中间人 C 用自己的私钥 SK_C 对 R_B 加密后发回给 B, 使 B 误以为是 A 发来的。A 收到 R_B 后也用自己的私钥 SK_A 对 R_B 加密后发回给 B, 但中途被 C 截获并丢弃。B 向 A 索取其公钥, 这个报文被 C 截获后转发给 A。

C 把自己的公钥 PK_C 冒充是 A 的公钥发送给 B, 而 C 也截获到 A 发送给 B 的公钥 PK_A 。

B 用收到的公钥 PK_C (以为是 A 的) 对数据 DATA 加密, 并发送给 A。C 截获后用自已的私钥 SK_C 解密, 复制一份留下, 然后再用 A 的公钥 PK_A 对数据 DATA 加密后发送给 A。

A 收到数据后,用自己的私钥 SK_A 解密,以为和 B 进行了保密通信。其实, B 发送给 A 的加密数据已被中间人 C 截获并解密了一份,但 A 和 B 却都不知道。

由此可见,公钥的分配以及认证公钥的真实性也是一个非常重要的问题。关于这点我们在后面(7.4.2 节)还要讨论。

7.4 密钥分配

由于密码算法是公开的,网络的安全性就完全基于密钥的安全保护上。因此在密码学中出现了—个重要的分支——**密钥管理**。密钥管理包括:密钥的产生、分配、注入、验证和使用。本节只讨论密钥的分配。

密钥分配(或密钥分发)是密钥管理中最大的问题。密钥必须通过最安全的通路进行分配。例如,可以派非常可靠的信使携带密钥分配给互相通信的各用户。这种方法称为**网外分配方式**。但随着用户的增多和网络流量的增大,密钥更换频繁(密钥必须定期更换才能做到可靠),派信使的办法已不再适用,而应采用**网内分配方式**,即对密钥自动分配。

7.4.1 对称密钥的分配

对称密钥分配存在以下两个问题。

第一,如果 n 个人中的每一个需要和其他 $n-1$ 个人通信,就需要 $n(n-1)$ 个密钥。但每两人共享一个密钥,因此密钥数是 $n(n-1)/2$ 。这常称为 n^2 问题。如果 n 是个很大的数,所需要的密钥数量就非常大。

第二,通信的双方怎样才能安全地得到共享的密钥呢?正是因为网络不安全,所以才需要使用加密技术。但密钥又需要怎样传送呢?

目前常用的密钥分配方式是设立**密钥分配中心 KDC (Key Distribution Center)**。KDC 是大家都信任的机构,其任务就是给需要进行秘密通信的用户临时分配一个会话密钥(仅使用一次)。在图 7-13 中假定用户 A 和 B 都是 KDC 的登记用户。A 和 B 在 KDC 登记时就已经在 KDC 的服务器上安装了各自和 KDC 进行通信的主密钥(master key) K_A 和 K_B 。为简单起见,下面在叙述时把“主密钥”简称为“密钥”。密钥分配分为三个步骤(如图 7-13 中带箭头直线上的①,②和③所示)。

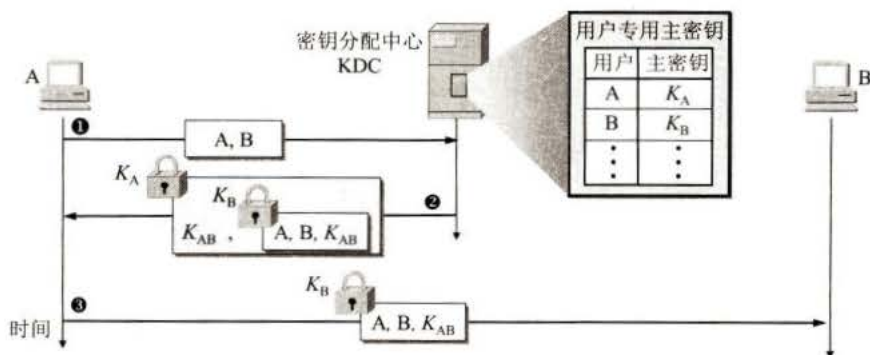


图 7-13 KDC 对会话密钥的分配

① 用户 A 向密钥分配中心 KDC 发送时用明文,说明想和用户 B 通信。在明文中给出

A 和 B 在 KDC 登记的身份。

② KDC 用随机数产生“一次一密”的会话密钥 K_{AB} 供 A 和 B 的这次会话使用，然后向 A 发送回答报文。这个回答报文用 A 的密钥 K_A 加密。这个报文中包含这次会话使用的密钥 K_{AB} 和请 A 转给 B 的一个票据(ticket)^①，该票据包括 A 和 B 在 KDC 登记的身份，以及这次会话将要使用的密钥 K_{AB} 。票据用 B 的密钥 K_B 加密，A 无法知道此票据的内容，因为 A 没有 B 的密钥 K_B ，当然 A 也不需要知道此票据的内容。

③ 当 B 收到 A 转来的票据并使用自己的密钥 K_B 解密后，就知道 A 要和他通信，同时也知道 KDC 为这次和 A 通信所分配的会话密钥 K_{AB} 。

此后，A 和 B 就可使用会话密钥 K_{AB} 进行这次通信了。

请注意，在网络上传送密钥时，都是经过加密的。解密用的密钥都不在网上传送。

KDC 还可在报文中加入时间戳，以防止报文的截取者利用以前已记录下的报文进行重放攻击。会话密钥 K_{AB} 是一次性的，因此机密性较高。而 KDC 分配给用户的密钥 K_A 和 K_B ，都应定期更换，以减少攻击者破译密钥的机会。

目前最出名的对称密钥分配协议是 Kerberos V5^② [RFC 4120, 4121, 建议标准]，是美国麻省理工学院(MIT)开发的。Kerberos 既是鉴别协议，同时也是 KDC，它已经变得很普及。Kerberos 使用比 DES 更加安全的高级加密标准 AES 进行加密。下面用图 7-14 介绍 Kerberos V4 的大致工作过程（其原理和 V5 大体一样，但稍简单些）。

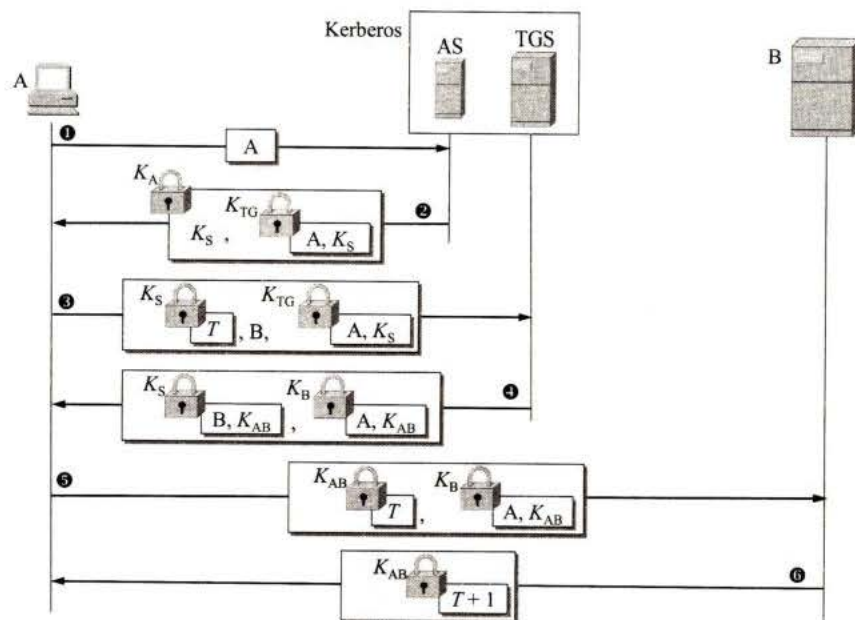


图 7-14 Kerberos 的工作原理

Kerberos 使用两个服务器：鉴别服务器 AS (Authentication Server)、票据授予服务器 TGS

① 注：目前在网络安全领域中 ticket 一词还没有标准译名，也有人译为“票”“执照”“票证”或“签条”。

② 注：Kerberos 是希腊神话中具有三个头的狗，它为 Hades（哈得斯，主宰阴间的冥王）看门。